Ministry of Education

University of Jeddah

Collage of Computer Science and Engineering

Course Project

CCSW 315 Software Process Models

Academic Year: Fall 2025

Final report

Atelier

Group 3		
Student Name	**Section**	**Id**
Shumokh Alsharif	SE3	2311165
Reman Alamoudi	SE3	2312582
Ghaida Almehmadi	SE1	2317174
Remas Alsulami	SE3	2310961
Marwa Hadaddi	SE	2310271

Important links:

Atelier Website

GitHub Repository

Design (Figma)

Agile planning (Trello)

Table of Contents

Sprint 0

Project Description

This project is about building a web platform that brings together artists, galleries, collectors, and people who love art. The platform will make it easy for artists to show their work, and for buyers to find, buy, or even request custom artworks. It is made to improve access and communication in the art community.

Problem definition

Today, many artists use social media or small local shows to share their work. This limits how many people see their art and lowers their chances of selling. Customers also find it hard to compare artworks or ask for special orders in a clear way. Because there is no one platform for both sides, artists and buyers often miss chances to connect.

Proposed Solution

The solution is to make a unified art platform that is simple and helpful for both artists and buyers. It will show products in a clear way, support safe online payments, and let customers save or request items. Artists will also have profile pages with information about them and their work. This will make it easier for buyers to trust and connect with them. Overall, the platform will help both sides by opening new chances for buying, selling, and working together in art.

Scope

A web platform for art enthusiasts, collectors, and galleries to browse artisan products, place orders, pre-orders, or commissions, and explore artisan profiles. The system includes a shopping cart, multiple payment options, and works on different devices and browsers.

Users

- Art Enthusiasts
- Art Collectors
- Galleries / Organizations
- Artisans

Product Backlog

Functional Requirements:

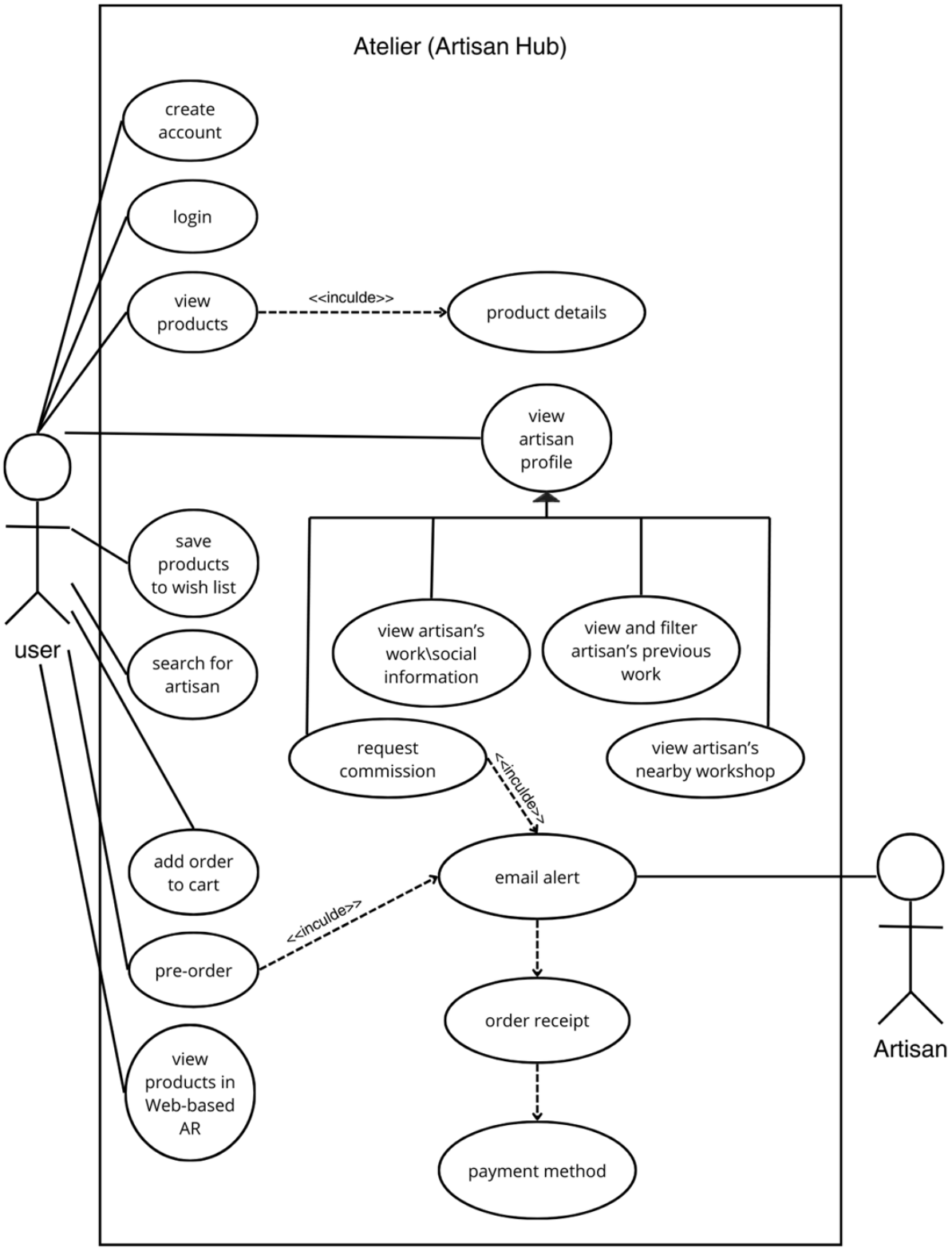
Req-ID	Description	Priority	Comment
FR-1	The system shall allow users to register and log in to their accounts	Low	
FR-2	The system shall display products in a card format within the main feed, with each card containing: - product photos. - The product price. - A description preview. - An "Add to Cart" button. - The artisan's profile picture and username	High	
FR-3	The system shall allow users to filter the main feed by category	High	
FR-4	The system shall provide a detailed product page which shall display: - All elements in FR-1 - A full product description - Detailed product dimensions. - Care instructions and cautions.	High	
FR-5	The system shall allow users to add products to a shopping cart.	High	
FR-6	The system shall group products by their respective shop (artisan) in the cart	Medium	
FR-7	The system shall calculate and display a subtotal for the items within each shop's grouping separately.	Low	A general total might be easier for users
FR-8	The system shall calculate the total of the cart.	High	
FR-9	The system shall provide a checkout process with the following payment methods: Apple Pay, debit card, credit card.	High	
FR-10	The system shall allow users to save products to a personal wishlist.	Low	
FR-11	The system shall allow users to navigate to an artisan's full profile by selecting the artisan's username	High	
FR-12	The system shall display an artisan's profile page containing: - The artisan's profile picture, and name. - A bio "About the Artist" - Links to the artisan's social media - A "Request Commission" button	High	

	- A gallery of the artisan's previous works.		
FR-13	The system shall provide filters for viewing artisan's work gallery "All Works" and "Available Works."	Low	
FR-14	The system shall provide a form for users to submit custom commission requests to an artisan, with the following fields: product type, dimensions, and a detailed description and attached media.	High	
FR-15	The system shall automatically send an email alert to the artisan upon receiving a commission request	Medium	
FR-16	The system shall allow users to place orders for products within a pre-order collection	Low	
FR-17	The system shall implement a Web-based Augmented Reality (Web AR) feature to allow users to visualize products in their environment.	Medium	Compatibility challenges

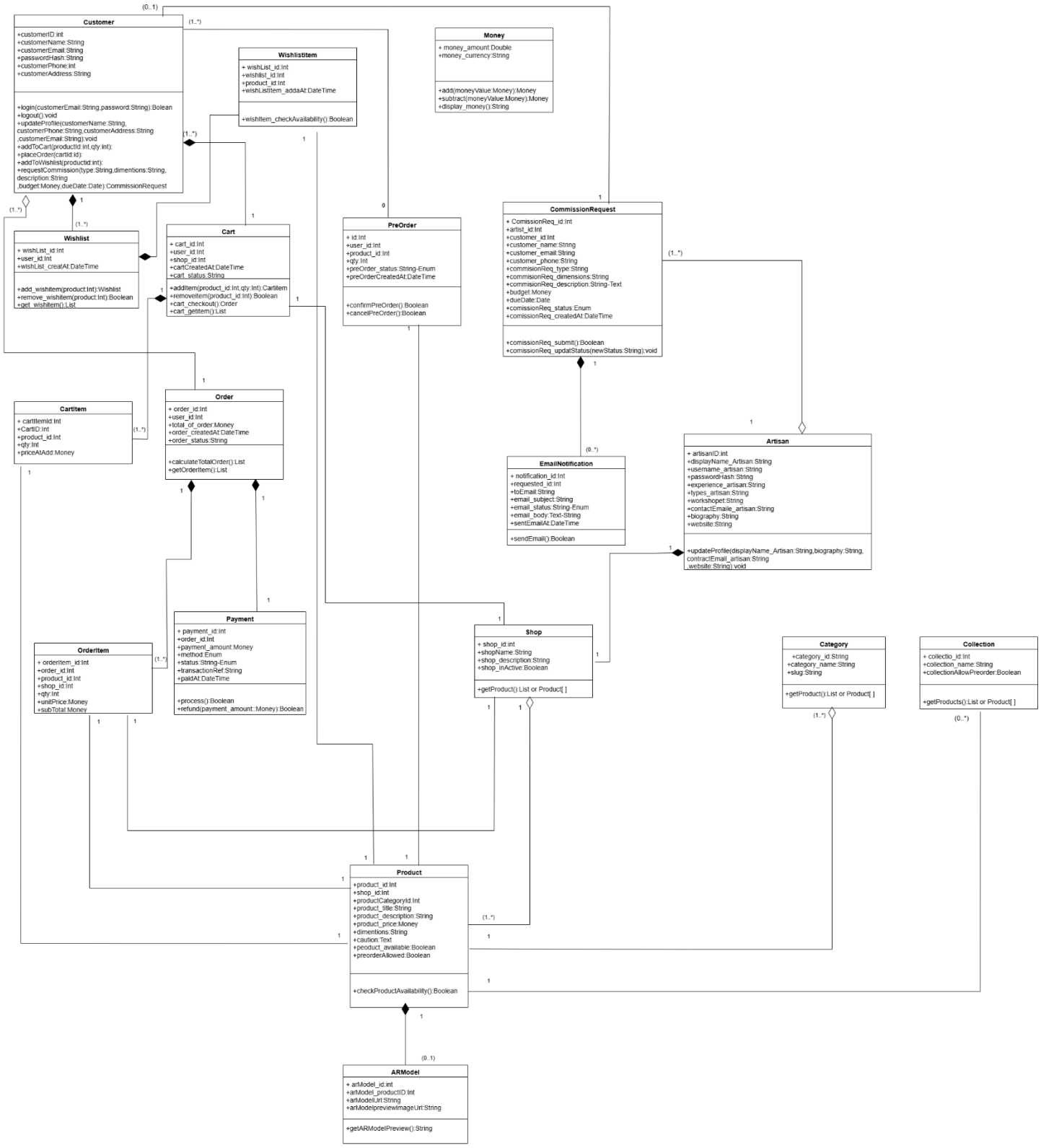
Non-functional Requirements:

Req-ID	Description	Priority	Comment
NFR-1	The main feed should load and be usable within 5 seconds	High	
NFR-2	The website should be useable on both mobile phones and desktop computers.	High	
NFR-3	User data, product information, and orders must be saved correctly and not be lost after a page refreshing	High	
NFR-4	The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.	Medim	
NFR-5	The system should support a minimum of 500 concurrent users performing typical activities	High	
NFR-6	The code should be consistently formatted and well documented	Medium	
NFR-7	The system data must be stored securely in the database	High	
NFR-8	Product images shall be optimized to minimize load time of 5 seconds	Low	
NFR-9	The system shall display images with minimum of 200PPI	High	

Use Case Diagram:



Class Diagram:



Tools To be Used:

Tool / Technology	Purpose
Visual Studio Code	IDE for writing frontend (HTML, CSS, JS) and backend code.
Git + GitHub	Version control for collaboration, commits, branches, and tracking changes.
Trello	Agile planning tool to manage sprint backlog, tasks, and progress.
Figma	Modeling/design tool for UI/UX mockups, wireframes, and style guide.
WhatsApp	Messaging & team coordination during development.
mongoDB	Database to store products, artisan profiles, orders; also provides authentication.
HTML5	Structure of web pages (frontend).
CSS3	Styling of web pages (colors, layout, responsiveness).
JavaScript (Vanilla)	Frontend interactivity, form validation.
Node.js	For backend server
Microsoft word	For documentation
Google meet	For virtual meetings

Sprint 1

1 Backlog (Name of functional requirement)

1.1 Components Name:

Browse main feed, browse collection, view product details

1.2 Functional Requirements:

FR-2: The system shall display products in a card format within the main feed

FR-3: The system shall allow users to filter the main feed by category

FR-4: The system shall provide a detailed product page which shall display

- Product photo
- The product price.
- An "Add to Cart" button.
- A full product description
- Detailed product dimensions.
- Care instructions and cautions.
- The artisan’s profile picture and username

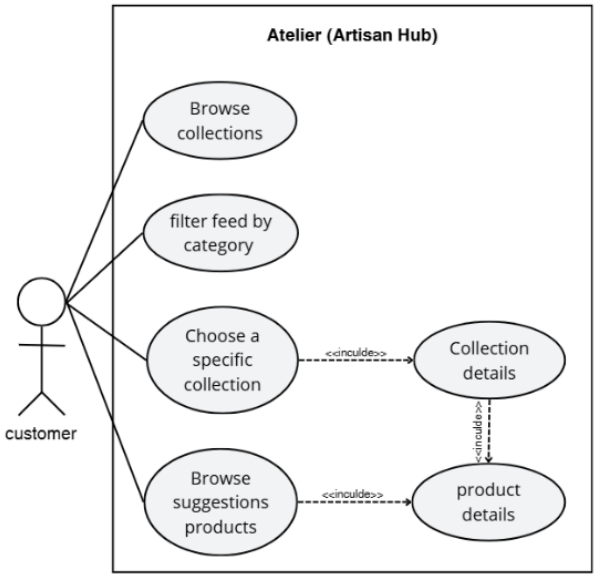
1.3 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

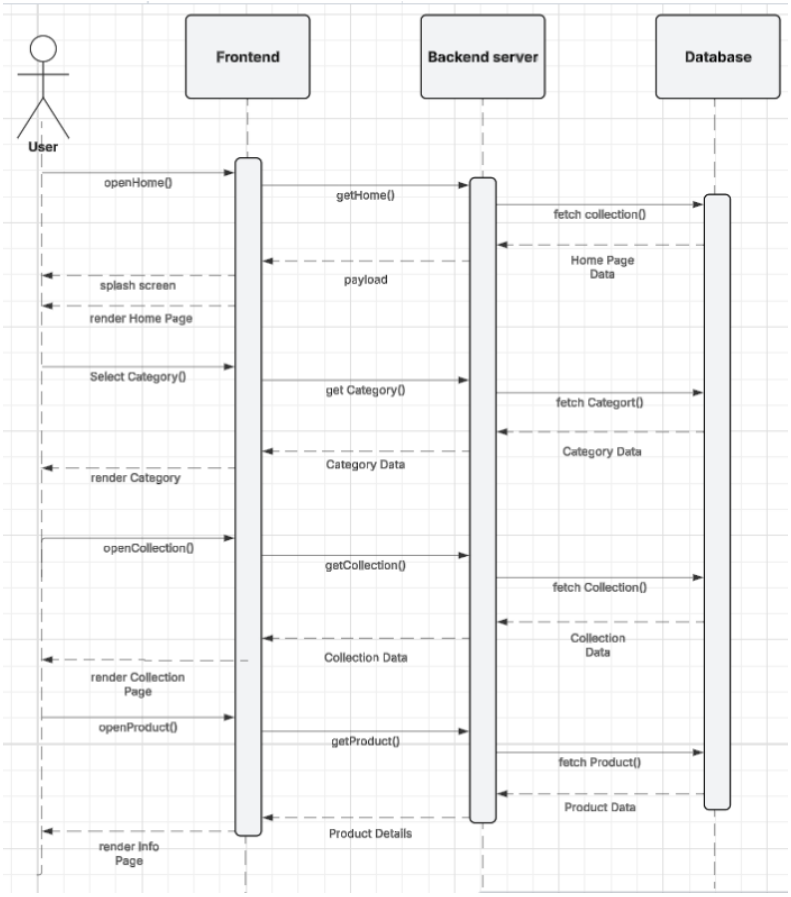
NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

2 Diagrams

2.1 Use case:



2.2 Sequence Diagram:



3 Stories:

3.1 Browse main feed

Component Name: Browse main feed

Story Name: Browse main feed

Story Sequence No: 01

Story Short Description: Users can browse the available collections and products.

Story Long Description: Users shall be able to browse products and featured collections which can be filtered according to the user's desired category. The data shall be sent from the database and displayed randomly to maximize product discovery for the user.

3.2 Browse collection

Component Name: Browse collection

Story Name: Browse collection

Story Sequence No: 02

Story Short Description: The system will display collection information.

Story Long Description: The user will navigate to the collection page when he clicks on a specific collection. The page will contain:

- collection name
- artist information (name, username and profile picture)
- all the products within the collection.

3.3 View product details

Component Name: View product details

Story Name: View product details

Story Sequence No: 02 or 03

Story Short Description: The system shall display product details.

Story Long Description: The user will navigate to the product page when he clicks on a specific product from the home page or the collection page. The page will display:

- the artist information (name, username, profile picture)
- product name
- Price
- dimensions
- category
- description

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
Stand up Meeting
[21 Sep 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- The high-fidelity design is completed and ready to hand off to developers
2- Next is:
   - Build the back-and server structure
   - Initiate the database with test data
   - Start with the front-end skeleton
3- So far none

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
progress check in
[25 Sep 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- **Initial pages are done, server and database successfully connected**
2- **Next is:**
   - Create the actual endpoint
   - Add responsive grid for home and collection pages
   - Switch front-end from hard coded data to fetching a Json file
   - Test server with postman
3- **Coordinating between each commit, push, pull to the repo**

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
progress check in
[29 Sep 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- **Front-end pages have all required elements, all back-end server endpoints work and respond correctly**
2- **Next is:**
   - Deploy the server
   - Connect the front-end with the back-end
   - Populating the database with real data
3- **The server denied front-end access so we couldn't load data from the database**

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
progress check in
[2 Oct 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- **Front-end is successfully connected to back-end, server is live**
2- **Next is:**
   - Add navigation between pages
   - Fix the fetching issues
   - Add an endpoint that lists all collections names
   - Adding a splash screen
3- **The collections endpoint needs improvement (provide the names listed & be filtered), artist data are not being fetched**

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
progress check in
[5 Oct 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- **Fetching artist data now works, the new endpoint is completed, navigation between pages is done**
2- **Next is:**
   - Complete the database
   - Merge the splash screen branch
   - Ensure visual consistency across all pages
   - Fix the painting and pottery category problem
3- **The intro keeps repeating every time the home page is loaded, not all categories are displayed**

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*1*
progress check in
[9 Oct 2025]

Sprint Duration: 3 weeks
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Browse main feed
- Browse collection
- View product details

Follow-up meeting questions:

1. What has been completed since the last meeting?
2. What are you going to be working on next?
3. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

1- All coding tasks are completed and the website fully functioning

2- Next is:

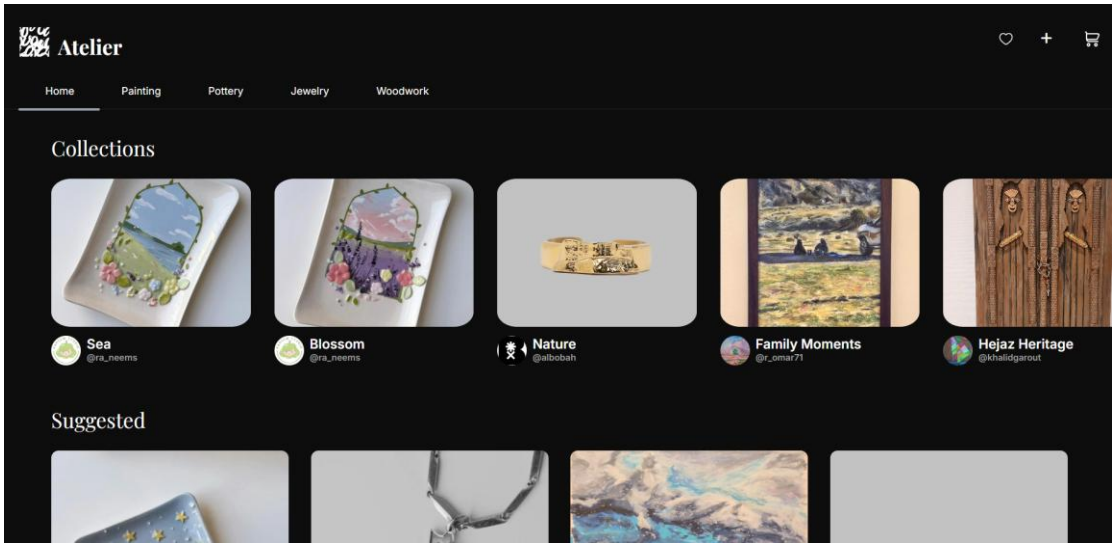
- Deploy the website
- Test on different devices
- Complete sprint documentation

3- So far none

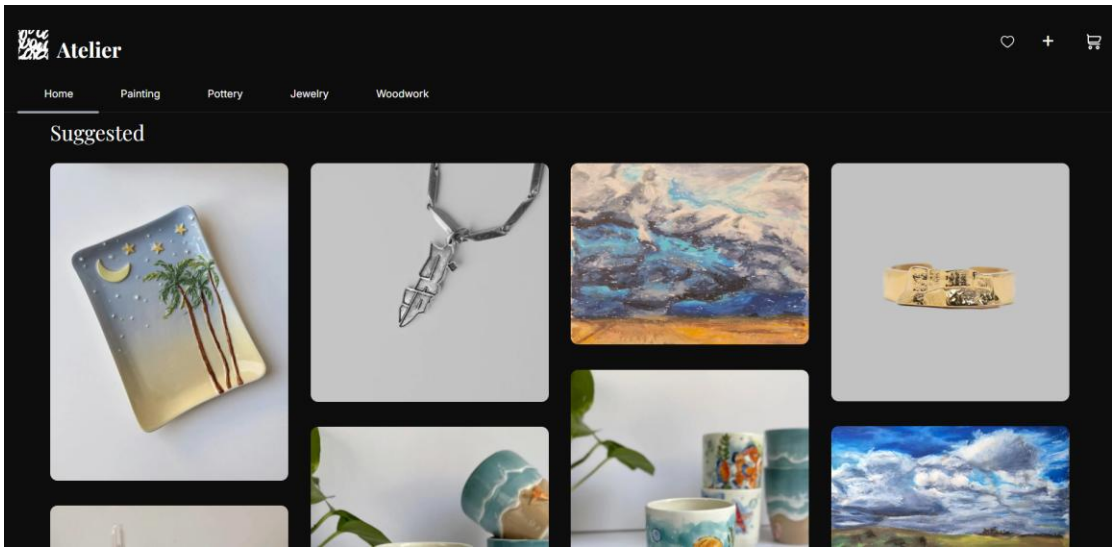
5 Implementation in Web browser (UI):

5.1 Splash screen

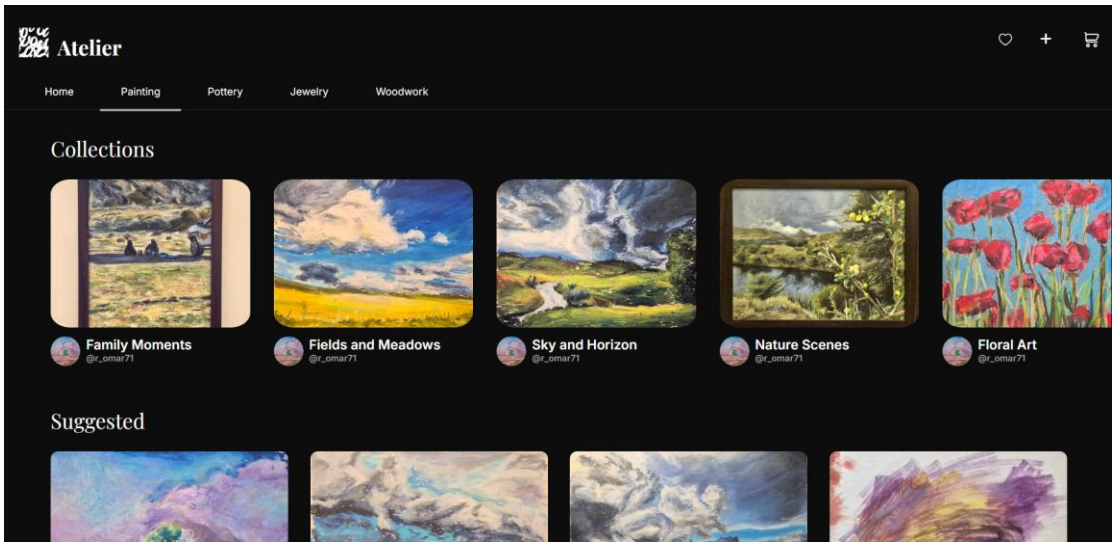
5.2 Home Page (collections)



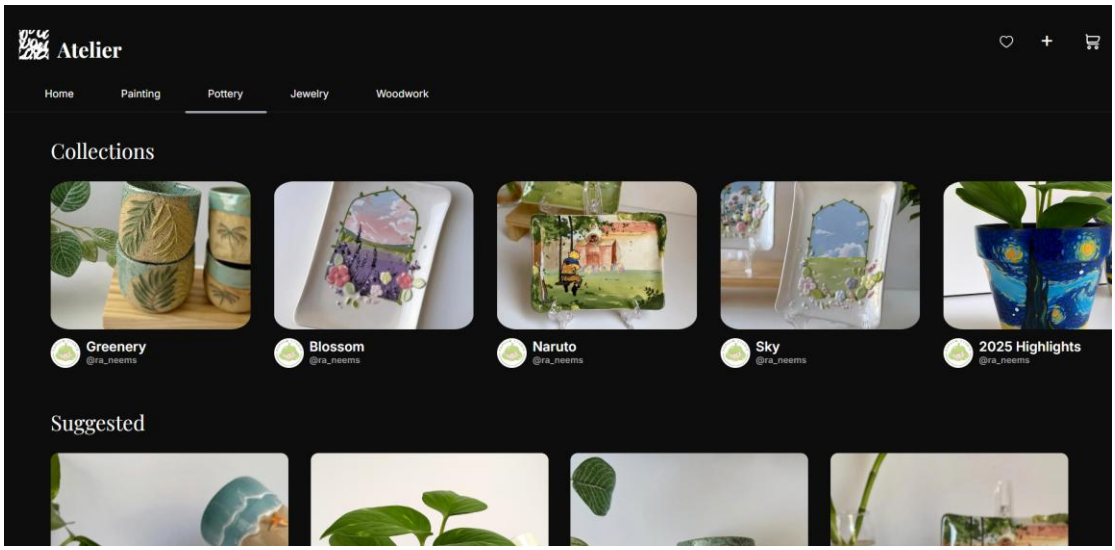
5.2 Home Page(suggestions)



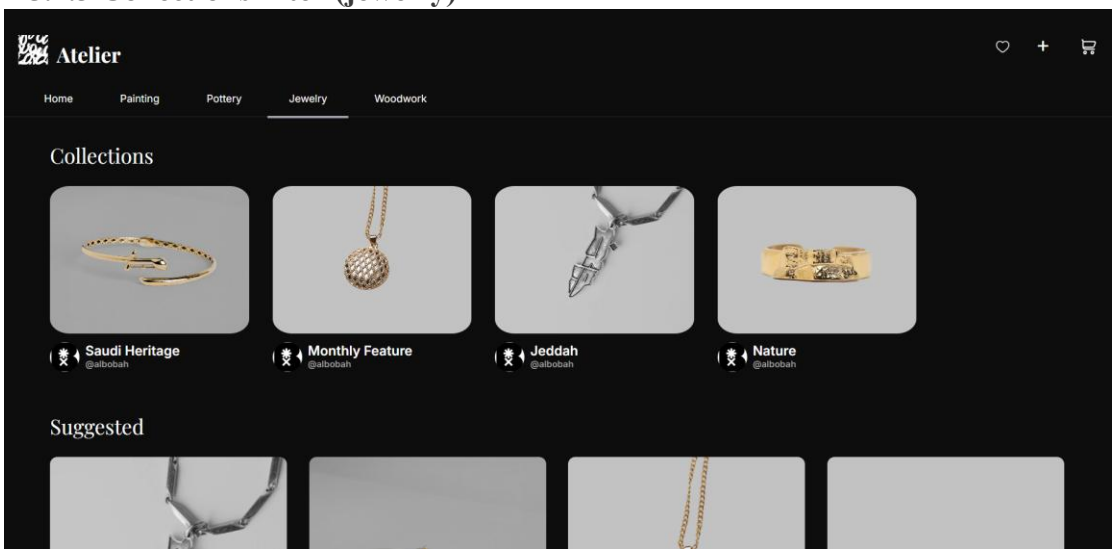
5.2.1 Collections filter (painting)



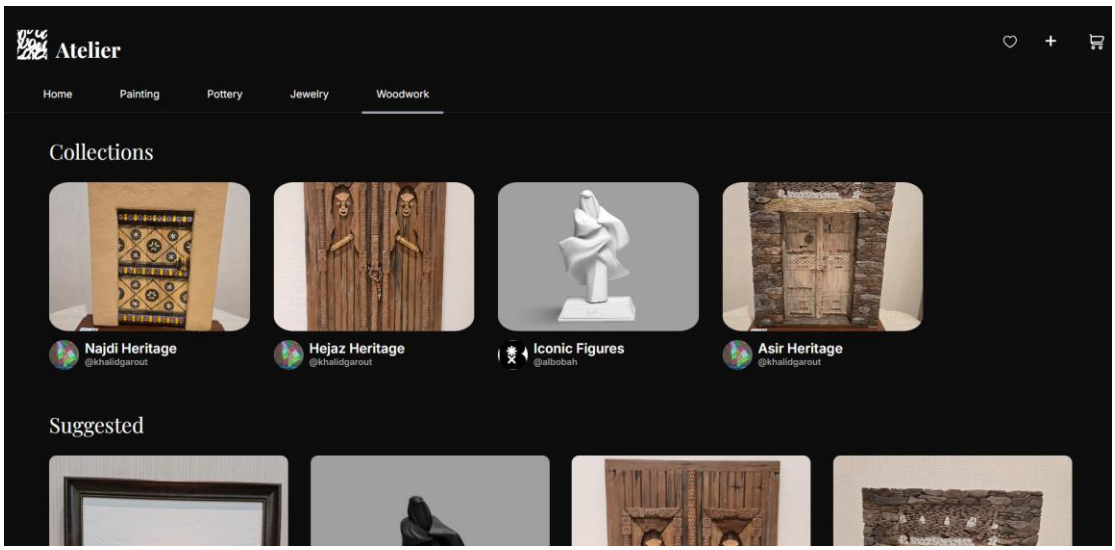
5.2.2 Collections filter (pottery)



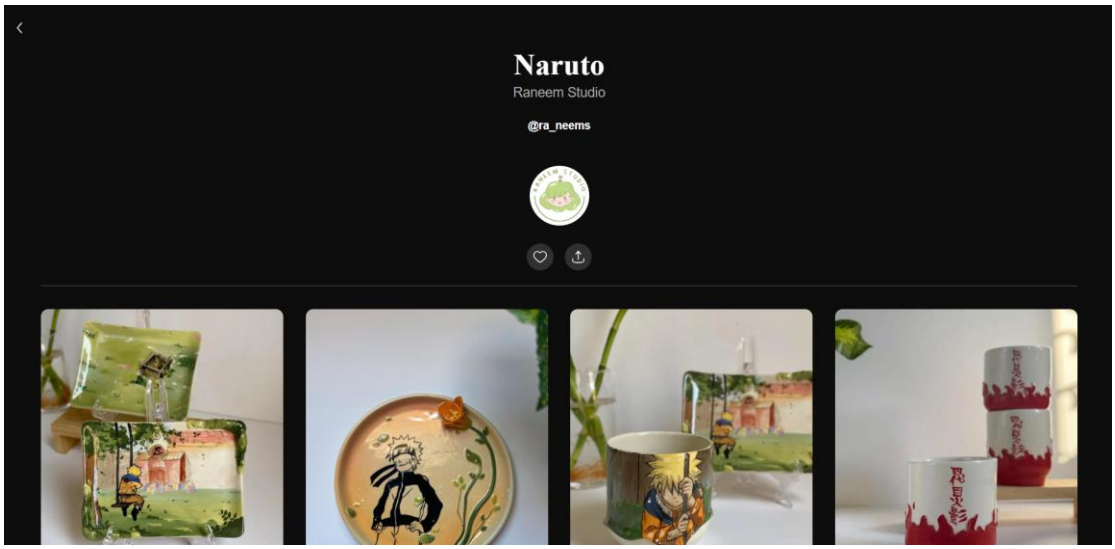
5.2.3 Collections filter (jewelry)



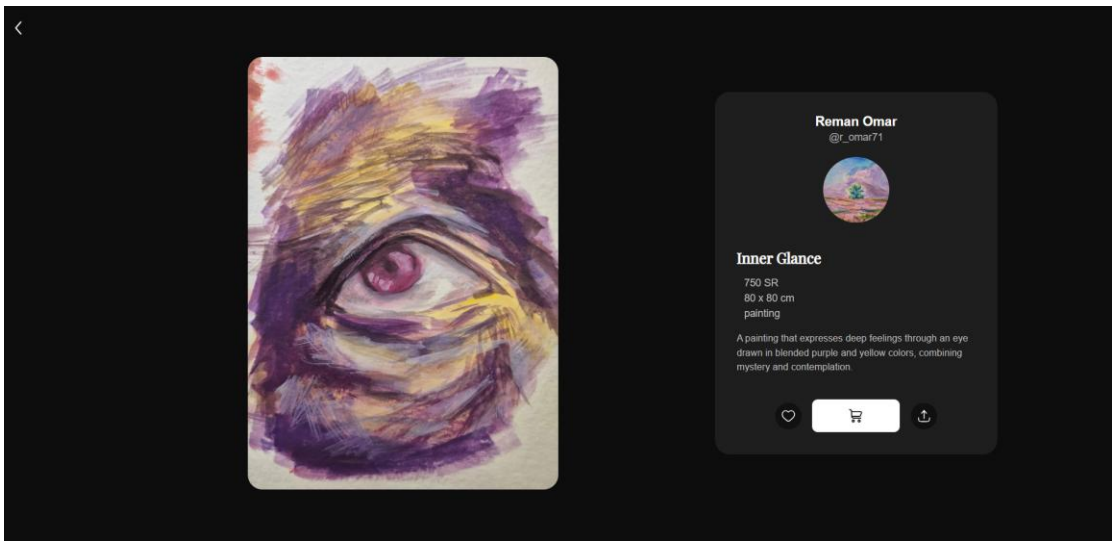
5.2.4 Collections filter (woodwork)



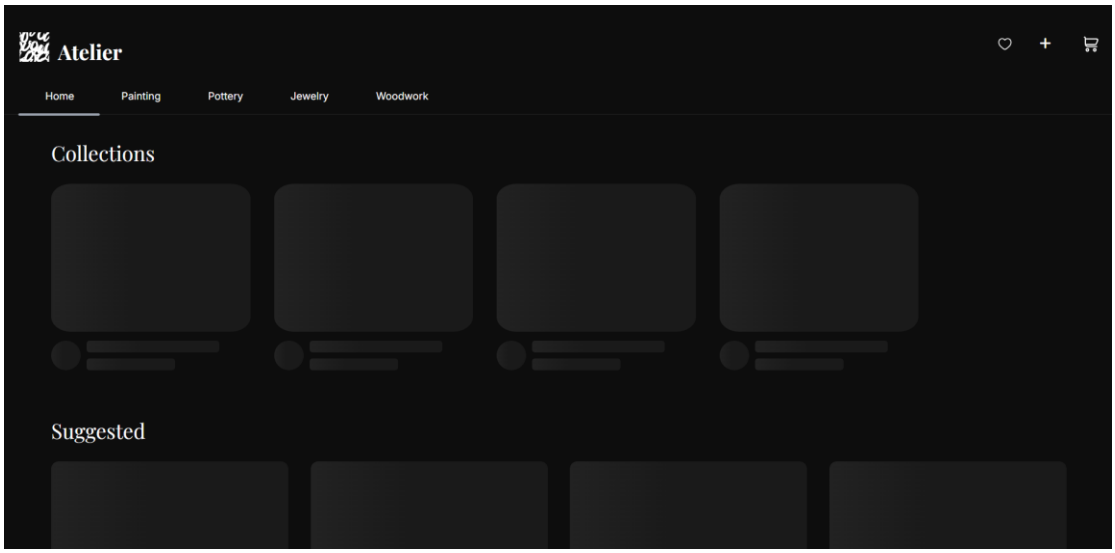
5.3 Collection Page



5.3 Product details Page

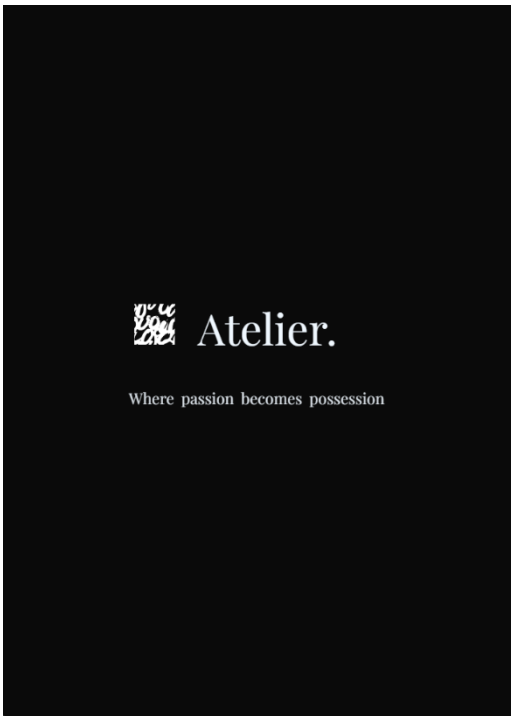


5.4 Loading skeleton

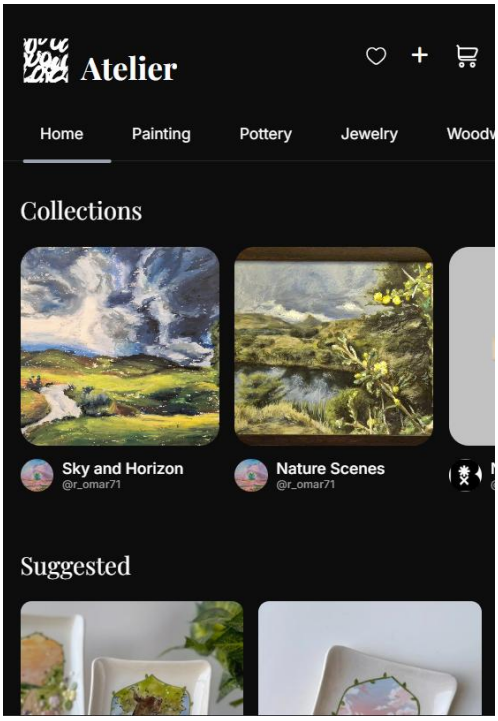


6 Implementation in Mobile (UI):

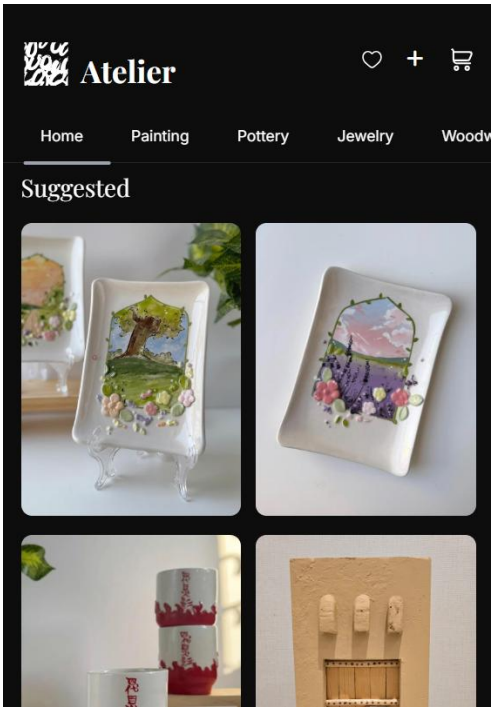
6.1 Splash screen



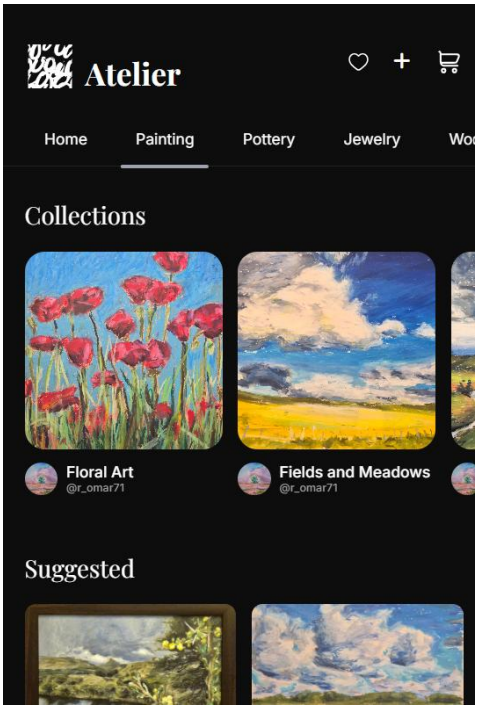
6.2 Home Page(collections)



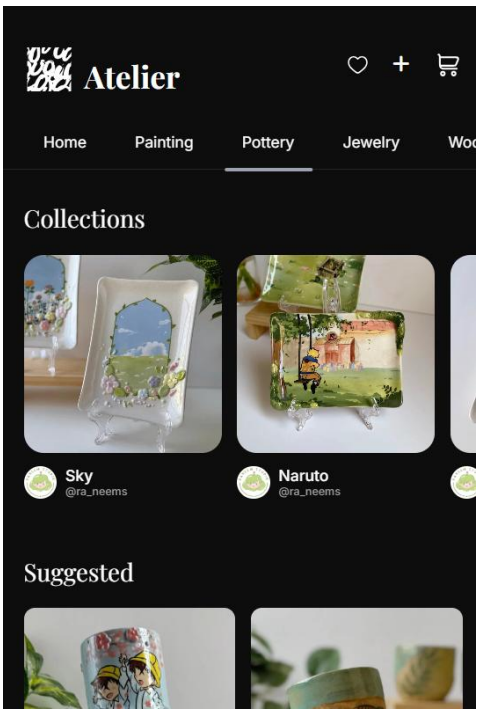
6.2 Home Page(suggestions)



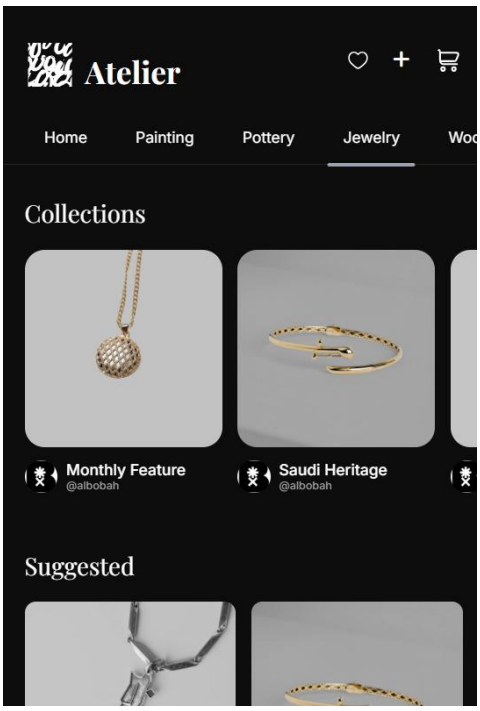
6.2.1 Collections filter (painting)



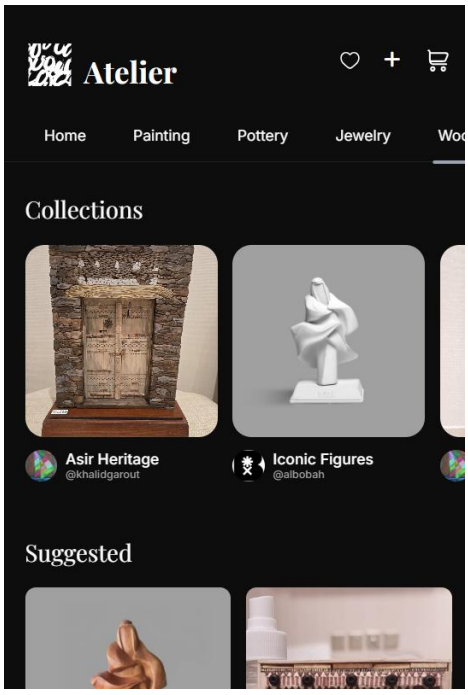
6.2.2 Collections filter (pottery)



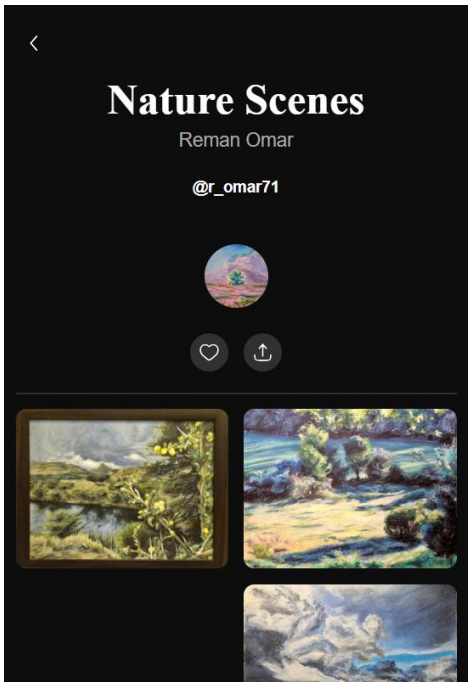
6.2.3 Collections filter (jewelry)



6.2.4 Collections filter (woodwork)



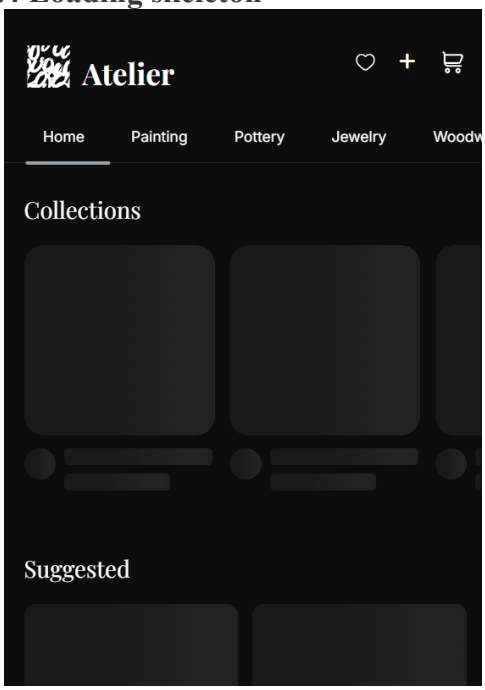
6.3 Collection Page



6.3.1 Product details Page

6.3.2 Product details Page

6.4 Loading skeleton



8 Test Cases

Test Case Name: Sprint 1-Browse main feed

Test Case Id: Atelier- Browse main feed

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User reloads page	Different products are displayed	Pass
TC002	User opens website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User clicks on a category	Feed filters collection and products by category	Pass
TC005	User opens the website	Splash screen displays	Pass
TC006	User hovers over a specific product	The display scale of the product increases	Pass
TC007	User hovers over a specific collection	Collection thumbnail changes to different image	Pass
TC008	User clicks on a specific product	Navigates to the product detail page	Pass
TC009	User clicks on a specific collection	Navigates to the collection page	Pass

Test Case Name: Sprint 1-Browse collections

Test Case Id: Atelier- Browse collections

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User navigates to collection page	All collection data is displayed	Pass
TC002	User open website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User hovers over a specific product	The display scale of the product increases	Pass
TC005	User clicks on a specific product	Navigates to the product detail page	Pass
TC006	User clicks on the back arrow	Navigates back to home page	Pass
TC007	User hovers over artist profile picture	The display scale of the picture will increase	Pass

Test Case Name: Sprint 1-View product details

Test Case Id: Atelier- View product details

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User navigates to product page	All product data is displayed	Pass
TC002	User opens website from different devices	The layout adjusts accordingly	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User hovers over artist profile picture	The display scale of the picture will increase	Pass
TC005	User clicks on the back arrow	Navigates back to home page	Pass

Sprint 2

1 Backlog (Name of functional requirement)

1.4 Components Name:

- Authentication
- Artisan Management
- Commission Management
- Wishlist

1.5 Functional Requirements:

FR-1: The system shall allow users to register and log in to their accounts

FR-9: The system shall allow users to save products to a personal wishlist.

FR-10: The system shall allow users to navigate to an artisan's full profile by selecting the artisan's username.

F**R-11:**The system shall display an artisan's profile page containing:

- The artisan's profile picture, and name.
- A bio "About the Artist"
- Links to the artisan's social media
- A "Request Commission" button
- A gallery of the artisan's previous works.

FR-12: The system shall provide filters for viewing artisan's work gallery "All Works" and "Available Works."

FR-13: The system shall provide a form for users to submit custom commission requests to an artisan, with the following fields: product type, dimensions, and a detailed description and attached media.

FR-14: The system shall automatically send an email alert to the artisan upon receiving a commission request

1.6 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

NFR-3: User data, product information, and orders must be saved correctly and not be lost after a page refreshing

NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

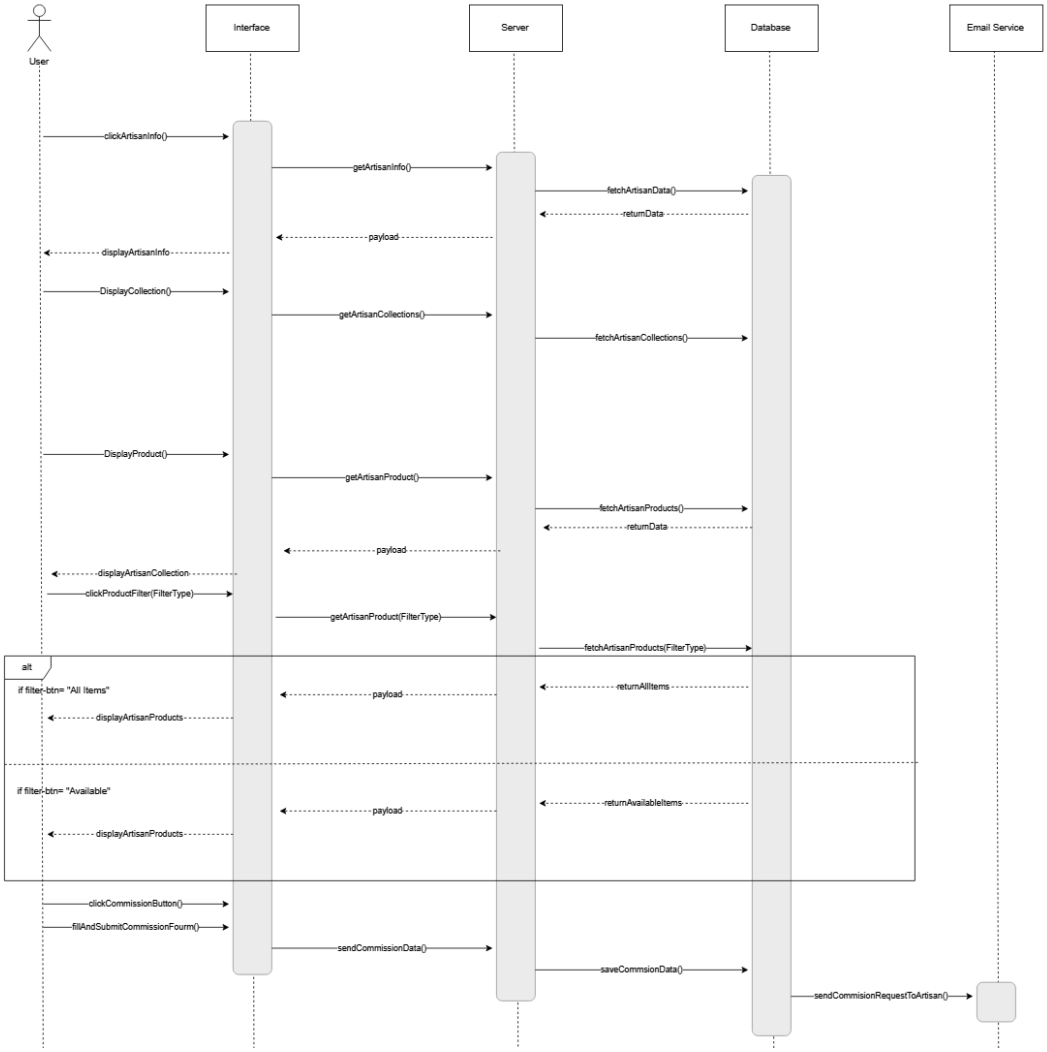
NFR-7: The system data must be stored securely in the database
NFR-8 : Product images shall be optimized to minimize load time of 5 second

2 Diagrams

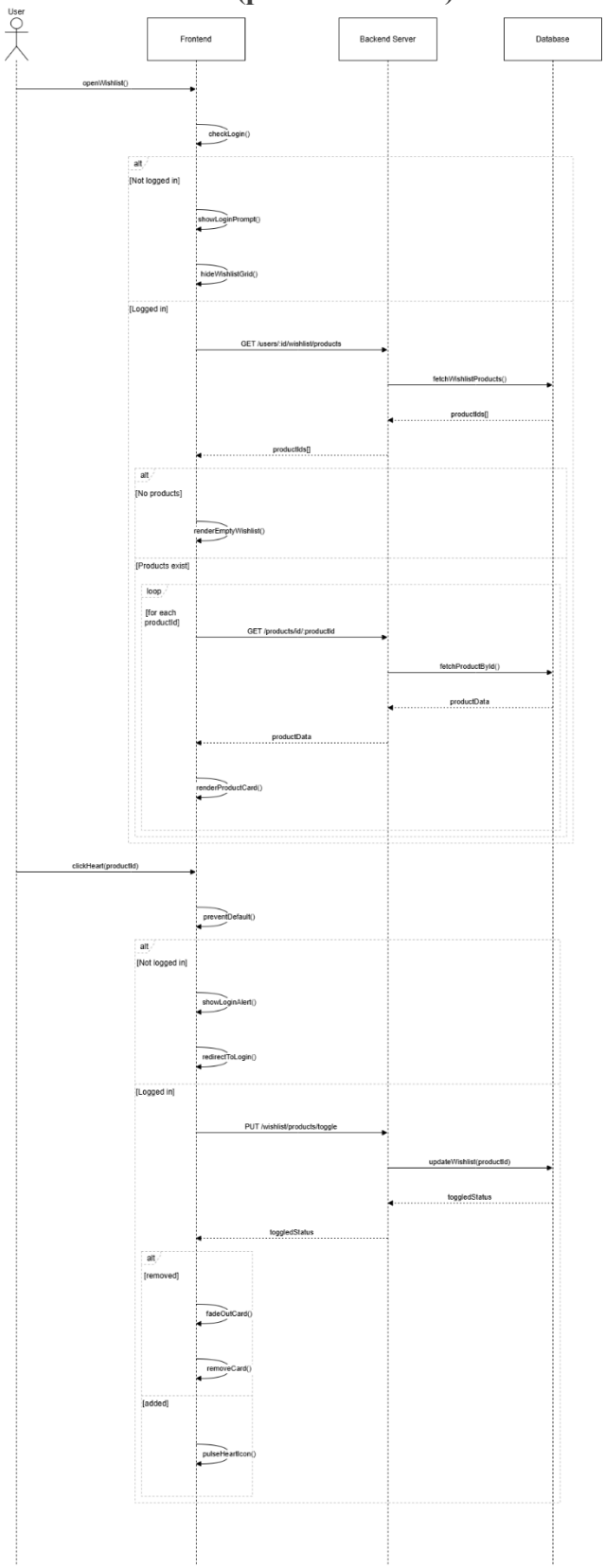
2.1 Use case:

2.2 Sequence Diagram

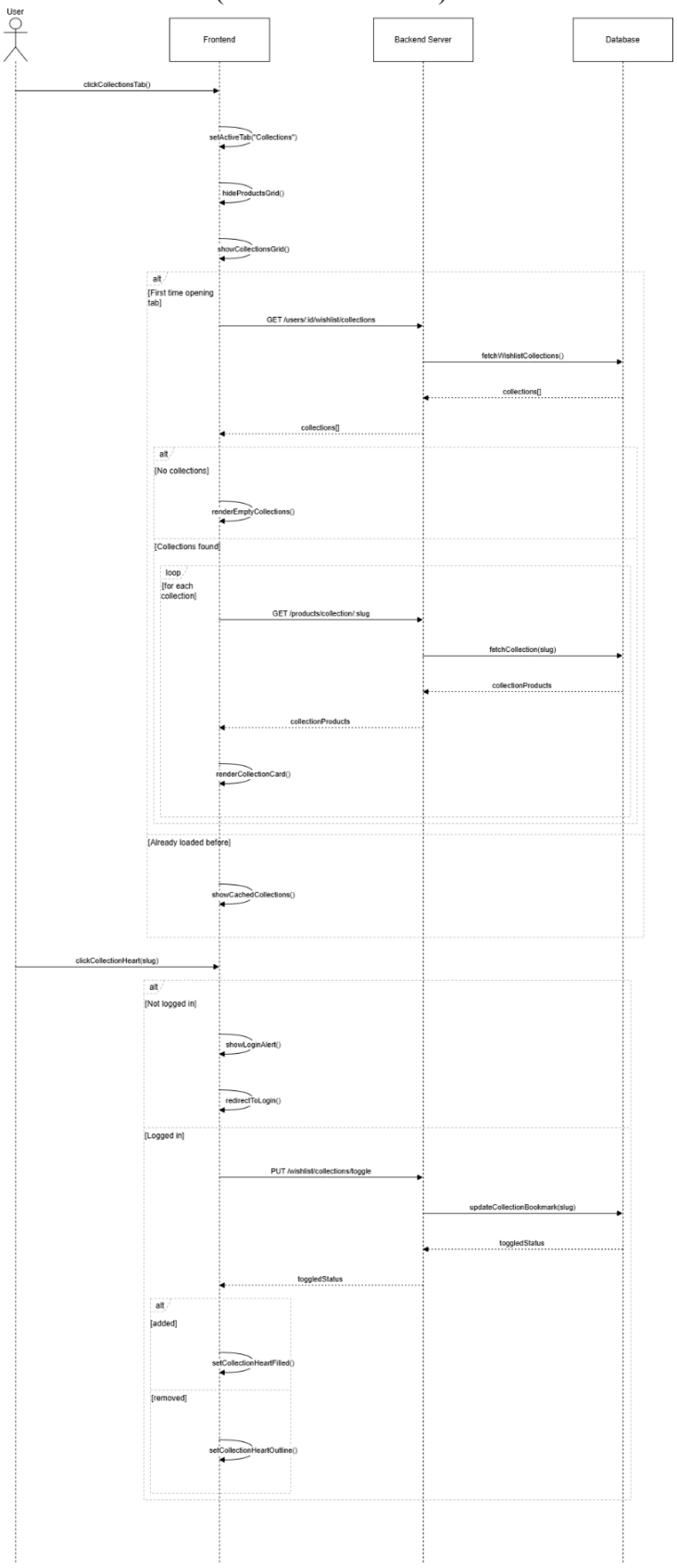
2.2.1 Profile Page



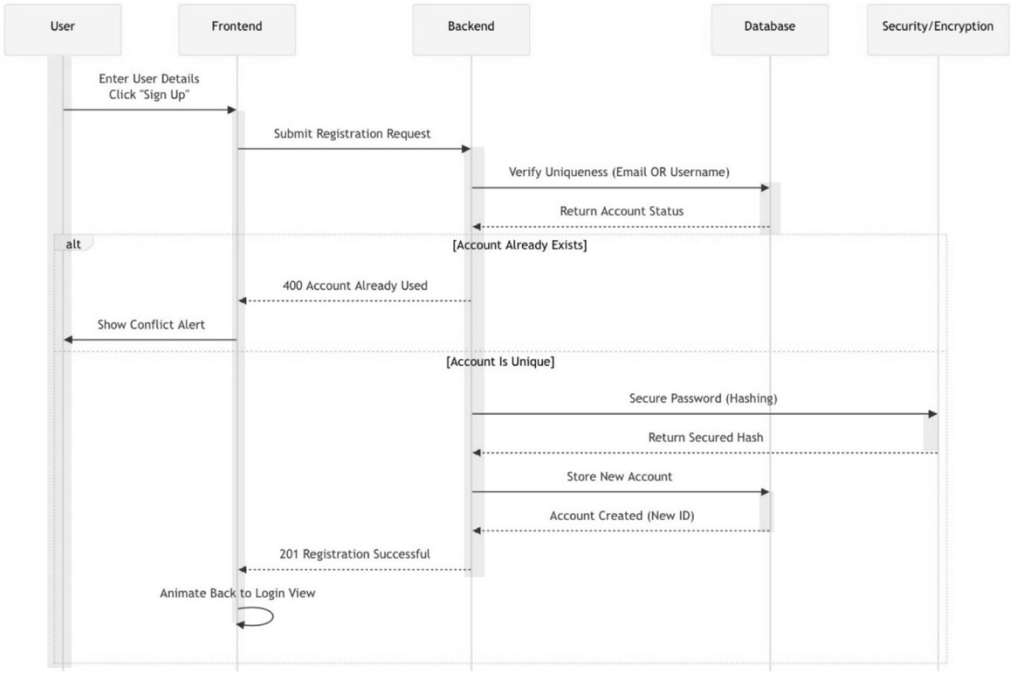
2.2.2 Wishlist: (product section)



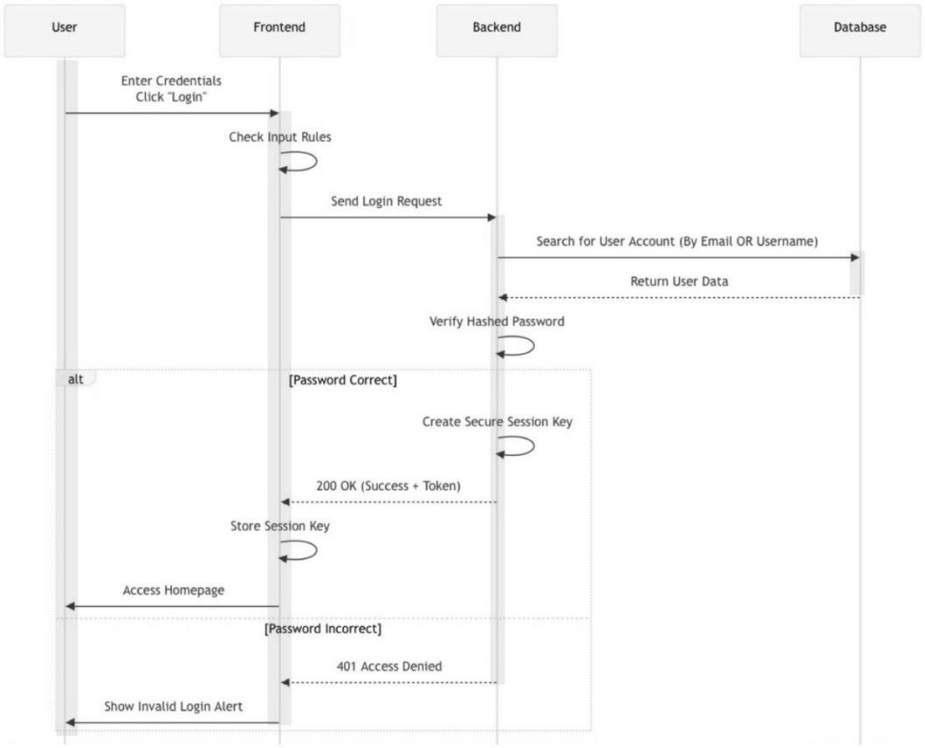
2.2.3 Wishlist: (collection section)



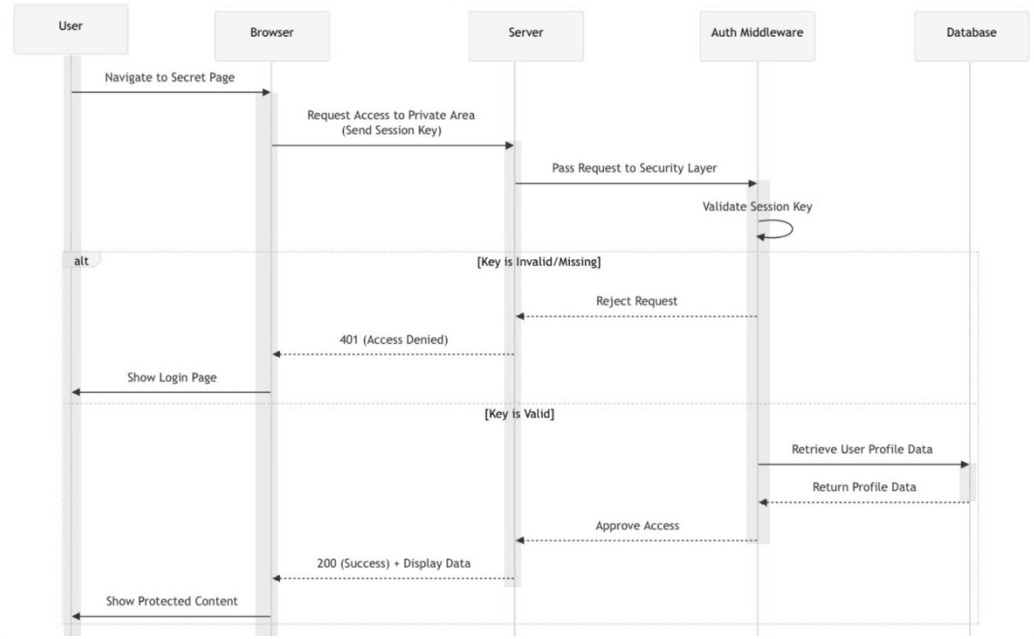
2.2.4 Sign Up:



2.2.5 Log In:



2.2.6 User Authorization:



3 Stories:

3.1 Login

Component Name: Authentication

Story Name: User Login

Story Sequence No: 01

Story Short Description: User logs into the system

Story Long Description: The user inputs email or username and password in order to authenticate and access their accounts.

3.2 Sign Up

Component Name: Authentication

Story Name: User Sign up

Story Sequence No: 02

Story Short Description: User creates new account

Story Long Description: A new user provides information (name , email, password and username) that are required to create a new account, allowing them to login and use full platform functionality.

3.3 View Artisan Profile

Component Name: Artisan Management

Story Name: View Artisan Profile

Story Sequence No: 03

Story Short Description: User views artisan details

Story Long Description: The user opens an artisan's profile to view their biography, artworks, social links and general information to discover more about the artisan.

3.4 Request Commission

Component Name: Commission Management

Story Name: Request Commission

Story Sequence No: 04

Story Short Description: User request custom artwork.

Story Long Description: The user submits a commission forum to a specific artisan by filling details allowing artisans to review and respond to clients.

3.5 View Wishlist

Component Name: Wishlist

Story Name: View Wishlist

Story Sequence No: 05

Story Short Description: User views saved products

Story Long Description: The user opens the wishlist page to view all the products they have already saved for later consideration or purchase.

3.6 Edit Wishlist

Component Name: Wishlist

Story Name: Add/Remove Wishlist Items

Story Sequence No: 06

Story Short Description: User manages Wishlist items.

Story Long Description: The user adds products to their Wishlist or removes them as they wish, allowing them to keep track of the items they are interested in.

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*2*
Stand up Meeting
[2 November 2025]

Sprint Duration: ...
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

4. What has been completed since the last meeting?
5. What are you going to be working on next?
6. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

4- Sprint tasks are distributed among team members
5- Designing the UI pages & start on the new server endpoint
6- So far none

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*2*
Progress Check In Meeting
[9 November 2025]

Sprint Duration: ...
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

7. What has been completed since the last meeting?
8. What are you going to be working on next?
9. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

7- Frontend pages and UI are fully built, new schema are added, mail service configured
8- Build the endpoints for the new pages, create a dynamic email template, link frontend with backend
9- No far none

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*2*
Progress Check In Meeting
[11 November 2025]

Sprint Duration: ...
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Login/Signup page
- Artisan profile
- Wishlist page
- Commission form

Follow-up meeting questions:

10. What has been completed since the last meeting?
11. What are you going to be working on next?
12. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:

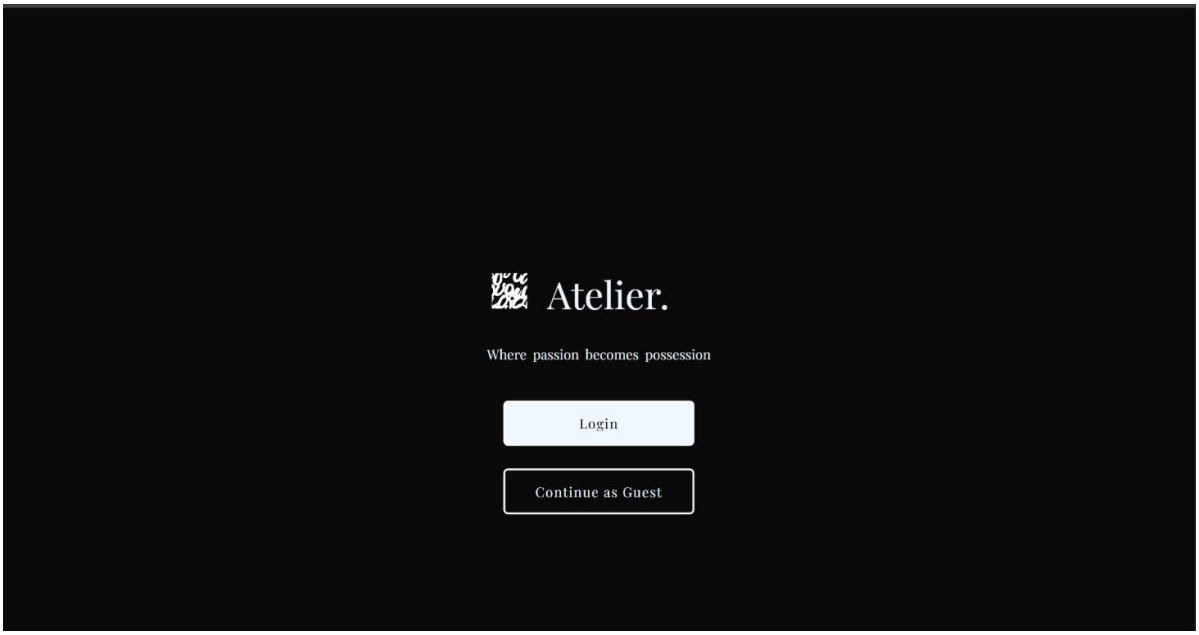
10- All new endpoints are added to the server, the frontend pages and complete, started configuring nodemailer

11- Link the commission form to the backend, adding navigation between pages, improve some of the endpoint's responses, link the login logic with the local storage

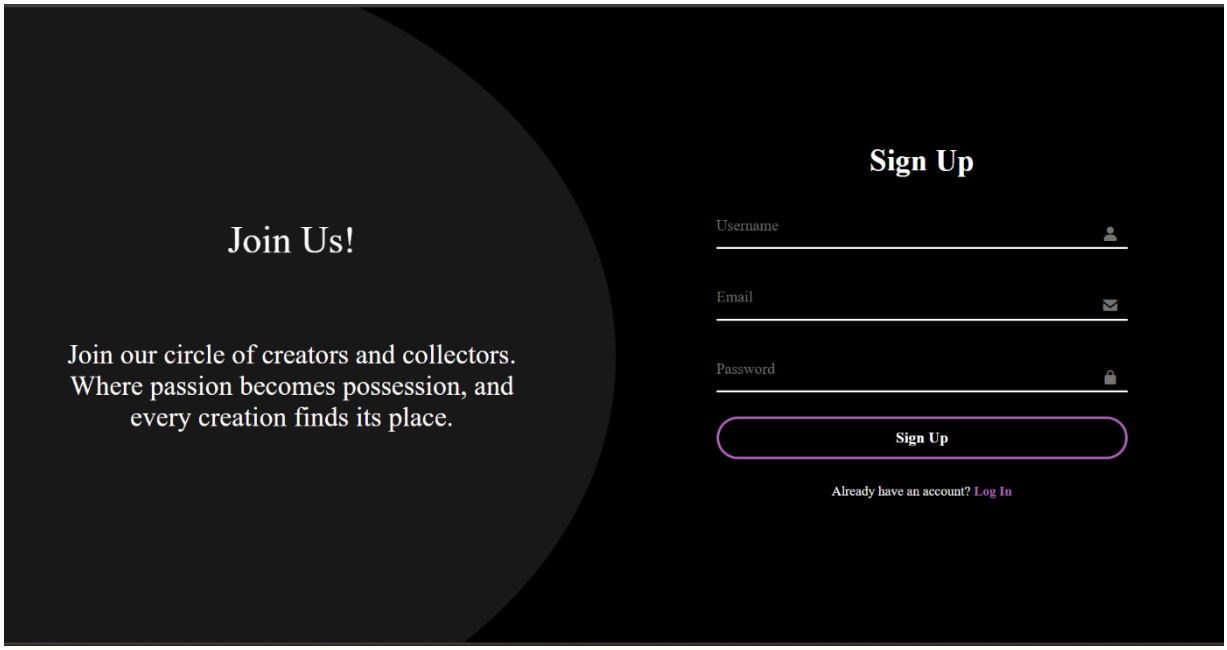
12- The current server host blocks SMTP protocols

5 Implementation in Web browser (UI):

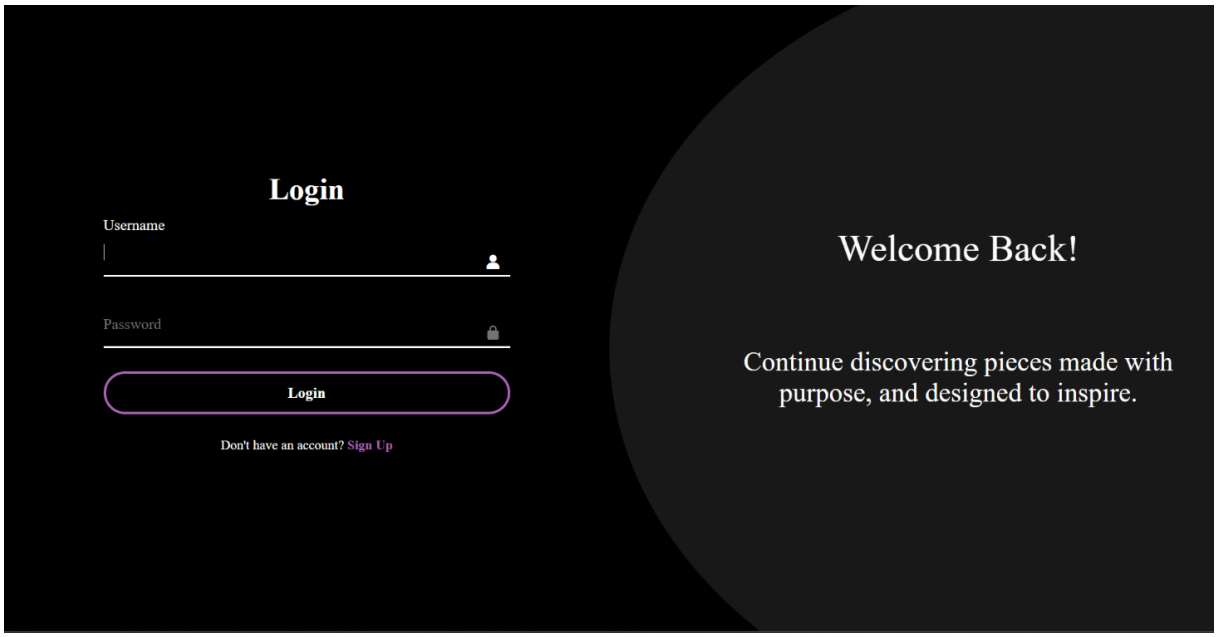
5.1 Welcome Page



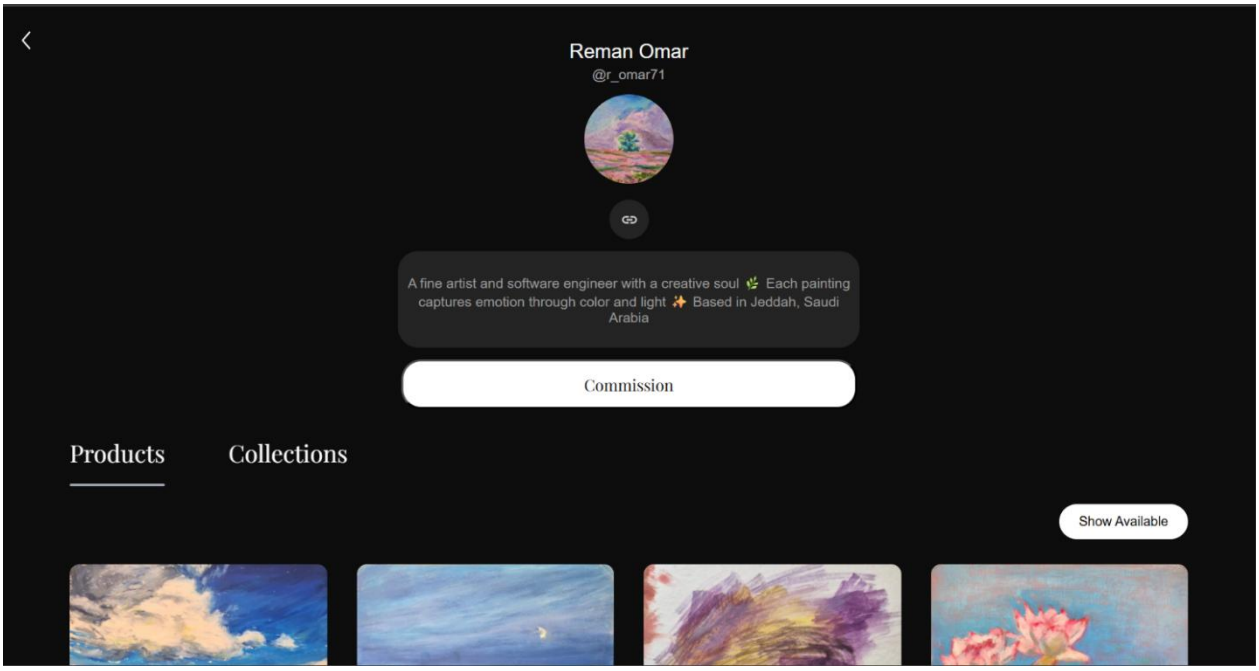
5.2 Sign in Page

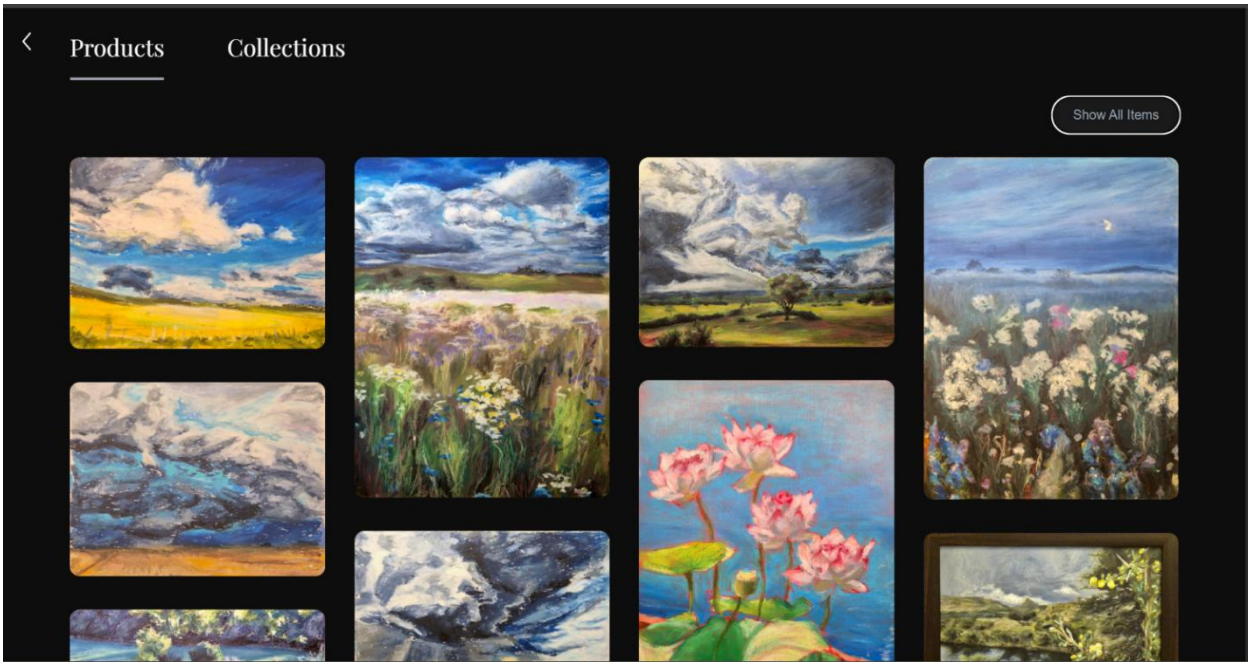


5.3Login Page

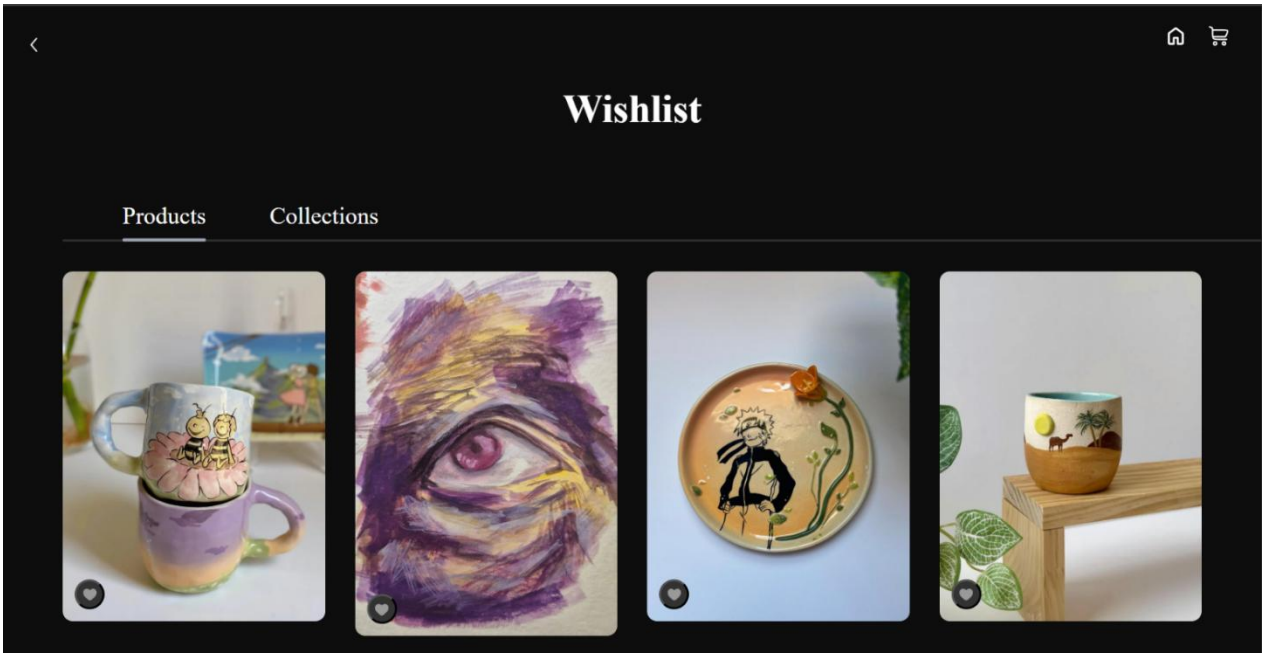


5.4 Profile Page

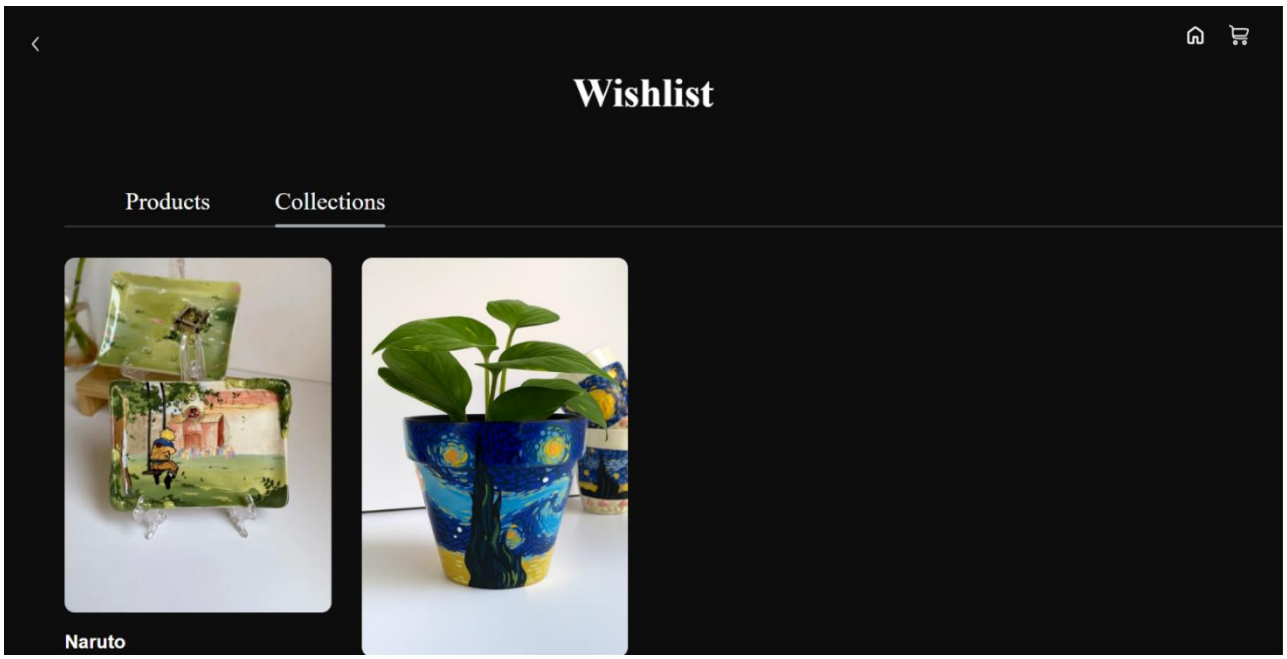




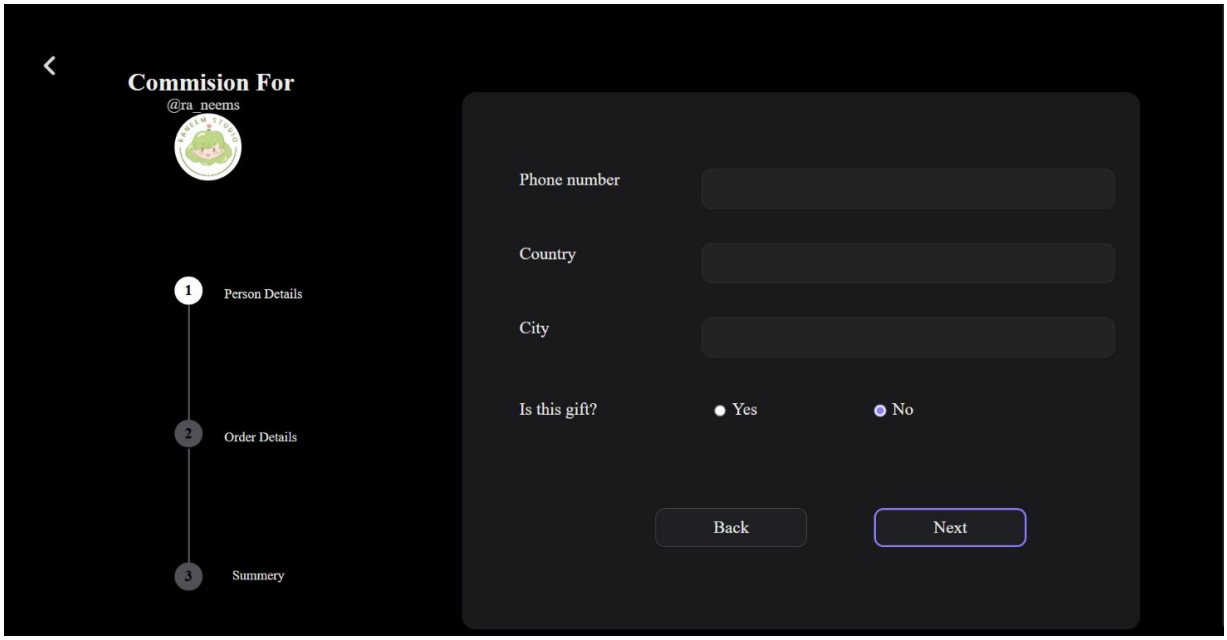
5.5 Wishlist Page



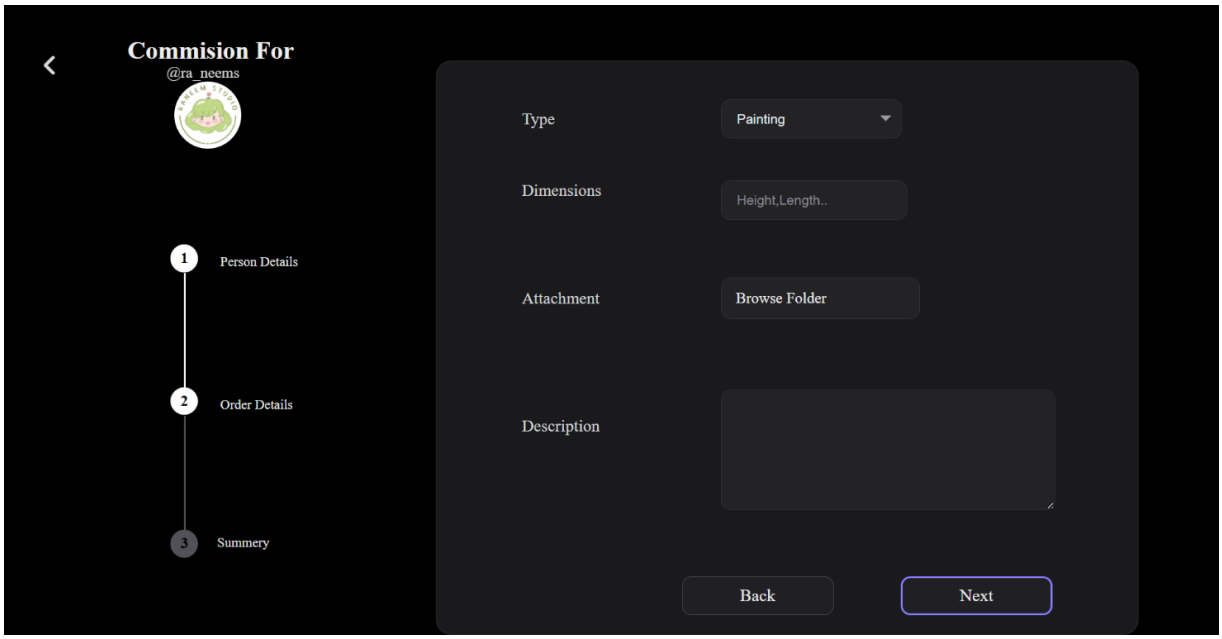
5.5.1 Wishlist Page



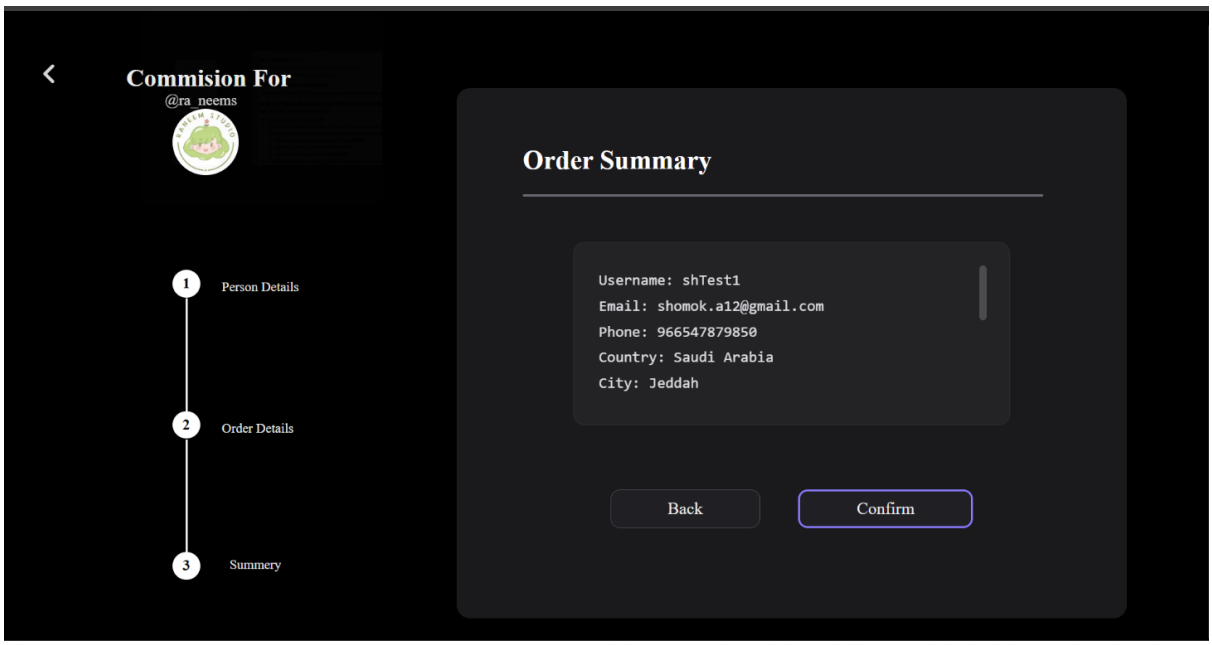
5.6 Commission Page

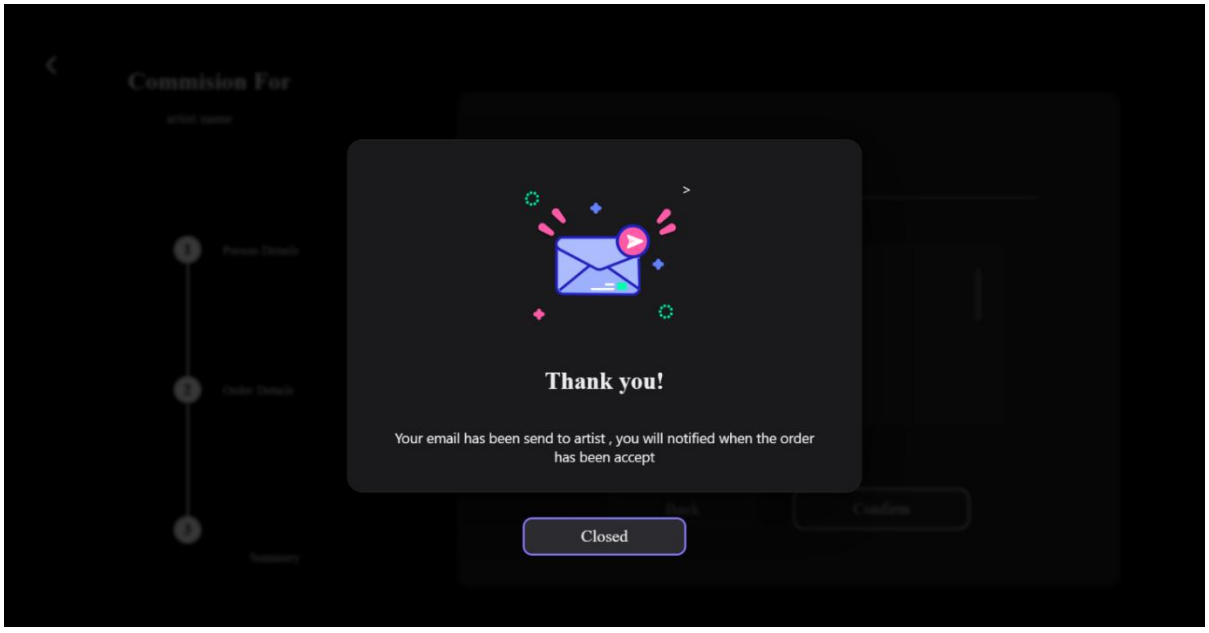


5.6.1 Commission Page



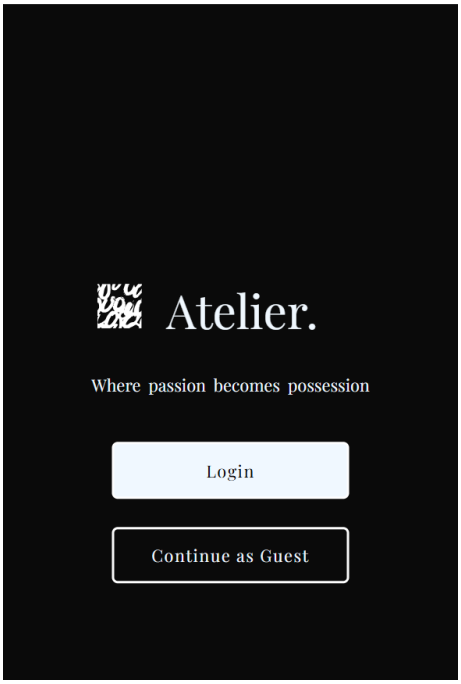
5.6.2 Commission Page



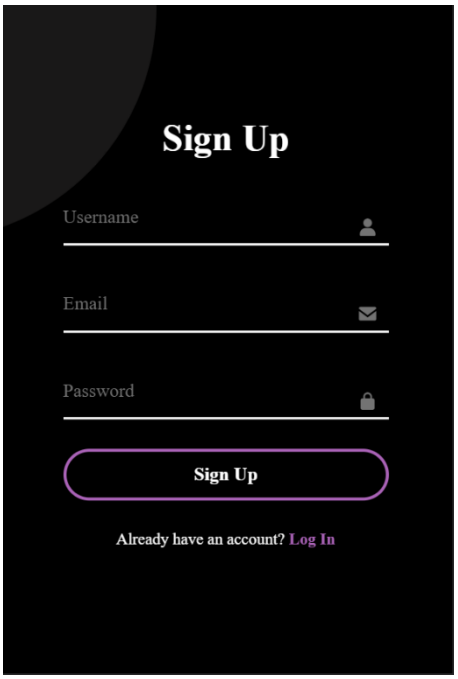


7 Implementation in Mobile (UI):

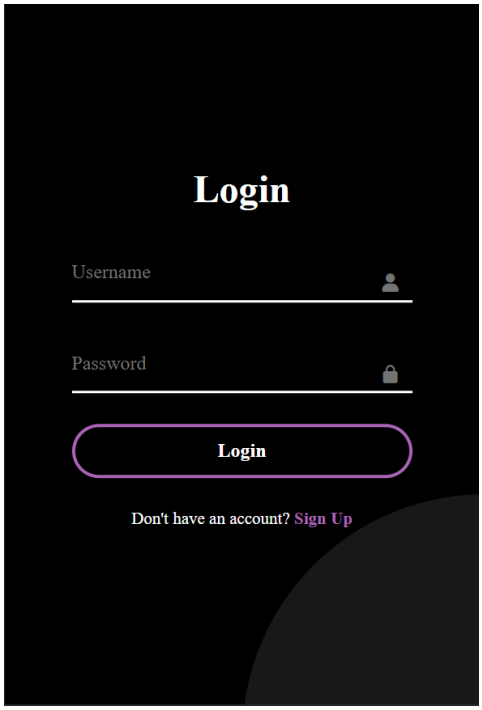
7.١ Welcome Page



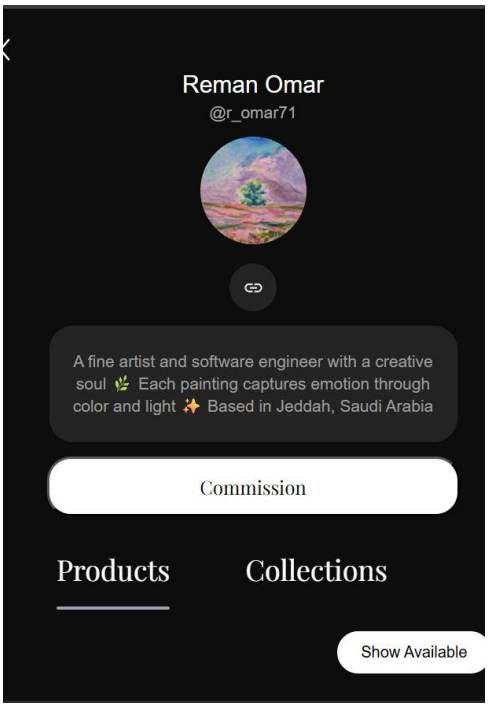
7.2 Sign in Page



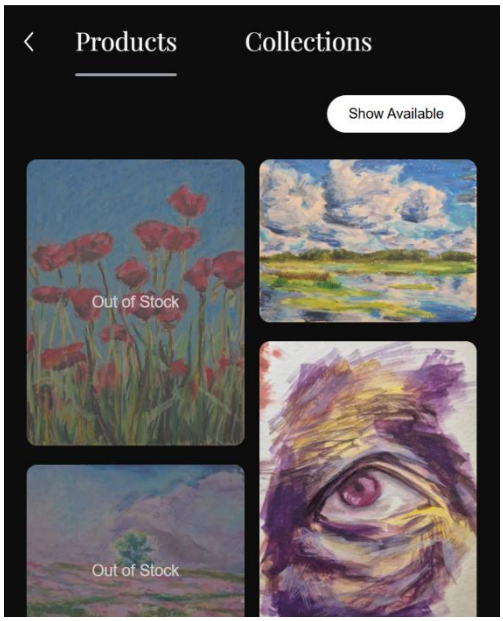
7.٣ Login Page



7.4 Profile Page

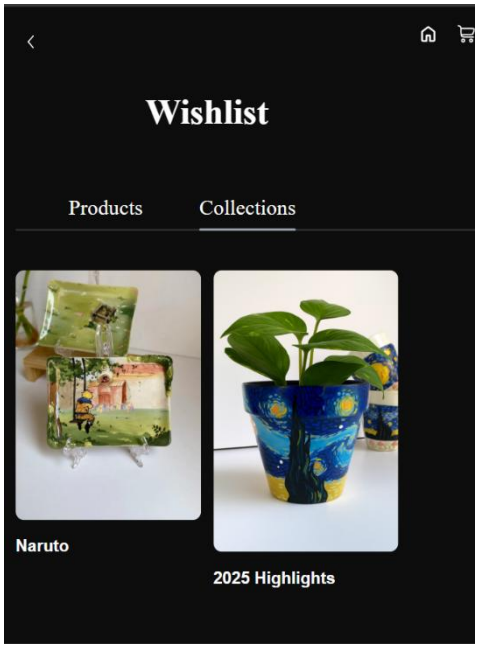


7.4.1 Profile Page

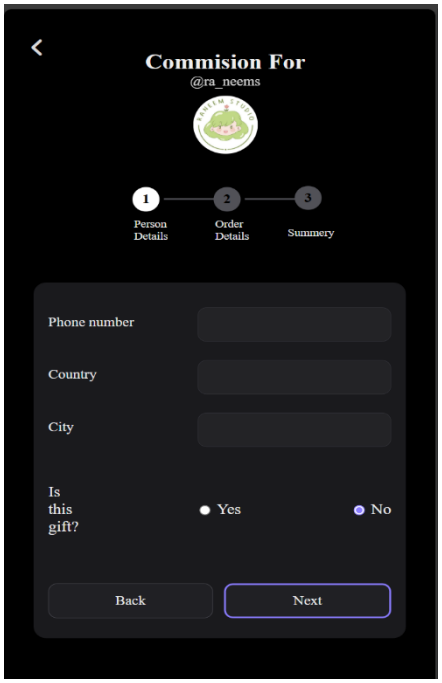


7.5 Wishlist Page

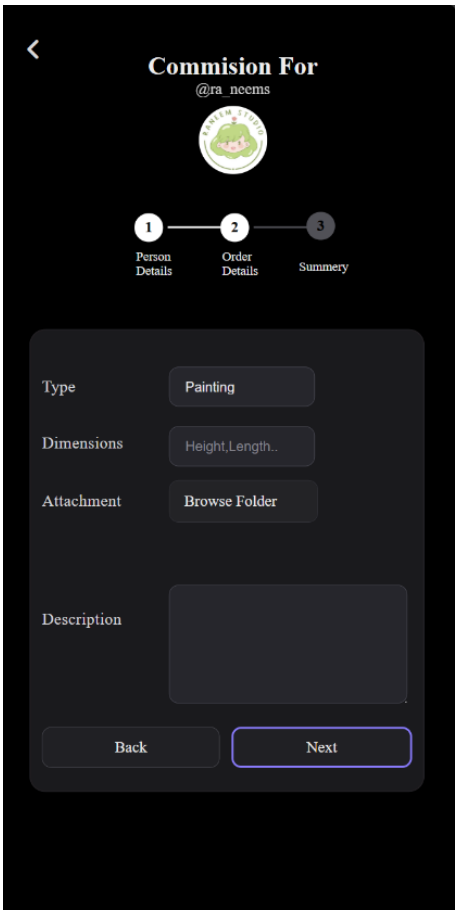
7.5.1 Wishlist Page



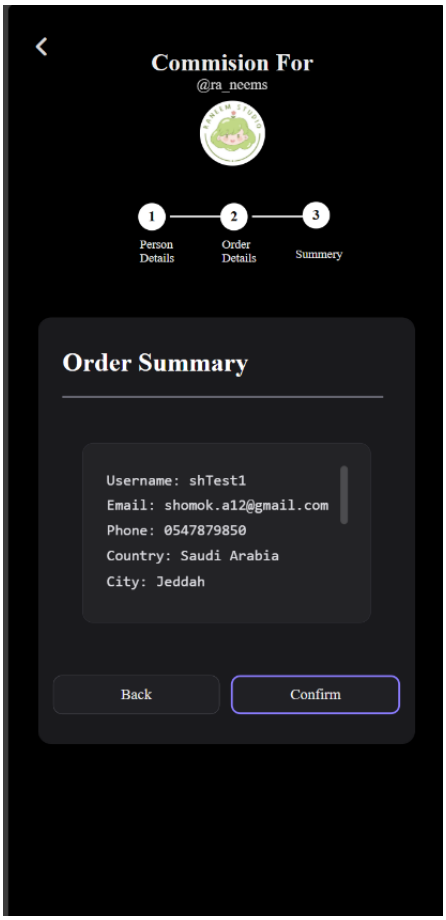
7.6 Commission Page



7.6.1 Commission Page



7.6.2 Commission Page



8 Test Cases

Test Case Name: Sprint 2- Log In

Test Case Id: Atelier – Log In

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User successfully logs in with valid Username and Password.	User is redirected to the main dashboard/feed.	Pass
TC002	User attempts to log in with an invalid Username but a valid Password.	An error message is displayed: “Invalid username or password.”	Pass
TC003	User attempts to log in with a valid Username but an invalid Password.	An error message is displayed: “Invalid username or password.”	Pass
TC004	User attempts to log in leaving the Username field empty.	A validation error/message prompts the user to enter the username.	Pass
TC005	User attempts to log in leaving the Password field empty.	A validation error/message prompts the user to enter the password.	Pass
TC006	User clicks the “Sign Up” or “Register Now” link on the Login page.	User is successfully navigated to the User Registration page.	Pass

Test Case Name: Sprint 2 - Registration

Test Case Id: Atelier – Registration

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User successfully registers with all valid and required information (Username, Email, Password).	User account is created, and the user is logged in/directed to a success page.	Pass
TC002	User attempts to register using an email address that is already in use.	An error message is displayed: “This email address is already in use.”	Pass
TC003	User attempts to register with an invalid email format (e.g., missing @ or .com).	An immediate validation error is displayed, guiding the user to correct the email format.	Pass
TC004	User attempts to register while leaving one of the required fields (e.g., Username) empty.	A validation error/message prompts the user to fill in the missing required field.	Pass
TC005	After successful registration, the user receives a confirmation dialog and clicks continue.	User is successfully navigated to the Login page.	Pass

Test Case Name: Sprint 2- Wishlist page

Test Case Id: Atelier - Wishlist page

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	Unregistered user opens the Wishlist page	Nothing is displayed and user will be prompted to log in first	Pass
TC002	Unregistered users try to add an item to the Wishlist	Product/collection won't be added and user will be prompted to log in first	Pass
TC003	Registered user clicks on the heart icon	The product/collection will be added to their Wishlist	Pass
TC004	Registered user opens the Wishlist page	All the products/collections on their Wishlist will be displayed	Pass
TC005	Registered user clicks on the heart icon for an item already in the Wishlist	Product/collection will be removed from the Wishlist	Pass
TC006	User change window size	The layout adjusts accordingly	Pass

Test Case Name: Sprint 2- commission

Test Case Id: Atelier - commission

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User clicks on the commission button	User goes to the commission form	Pass
TC002	User forgets a required field	The required field will vibrate, and the user won't continue until it is filled	Pass
TC003	User clicks on the next button	It takes them to the next part of the commission form	Pass
TC004	User clicks back	It takes them to the previous part of the form	Pass
TC005	User chooses to attach a file or a photo	The device file system is opened	Pass
TC006	User submits their commission	A confirmation message is displayed on screen	Pass
TC007	A new commission is received	Both the user and artisan get an email notification	Pass
TC008	User change window size	The layout adjusts accordingly	Pass

Test Case Name: Sprint 2- artist profile

Test Case Id: Atelier - artist profile

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User clicks on the artist's photo/username on the Home Page.	User is successfully navigated to the Artist Profile Page, displaying their information and works.	Pass
TC002	User clicks on the artist's photo/username on the Collection Page.	User is successfully navigated to the Artist Profile Page.	Pass
TC003	User clicks on the artist's photo/username on the Product Page.	User is successfully navigated to the Artist Profile Page.	Pass
TC004	User clicks on the artist's social media icon/link.	The artist's social media account opens (preferably in a new window/tab).	Pass
TC005	User hovers the mouse over an out-of-stock product.	The text "Out of Stock" appears, and the product image is overlaid with a slight gray transparency.	Pass
TC006	User clicks the "Availability" button.	The filter is applied, and only "Available" products are displayed.	Pass
TC007	User clicks the "Availability" button again.	The filter is removed, and "All products" (Available and Out of Stock) are displayed.	Pass
TC00٨	User clicks on any "product" on the artist's page.	User is navigated to the product's specific page (Product/Collection page).	Pass
TC00٩	User clicks the "Back" button from the product page.	User successfully returns to the Artist Profile Page.	Pass
TC0١٠	User clicks on the "Collections" button/word.	All of the artist's collections are displayed with their names.	Pass

TC0١١	User clicks on the “Commission” button.	User is navigated to the artist’s specific Commission request page.	Pass
TC0١٢	The Artist Profile Page loads.	The artist’s name, profile picture, and biography (Bio) are all displayed correctly.	Pass

Sprint 3

1 Backlog (Name of functional requirement)

1.1 Components Name:

- Cart
- Checkout
- Web-AR

1.2 Functional Requirements:

FR-5: The system shall allow users to add products to a shopping cart.

FR-6: The system shall group products by their respective shop (artisan) in the cart

FR-8: The system shall calculate the total of the cart.

FR-9: The system shall provide a checkout process with the following payment methods: Apple Pay, debit card, credit card.

FR-17: The system shall implement a Web-based Augmented Reality (Web AR) feature to allow users to visualize products in their environment.

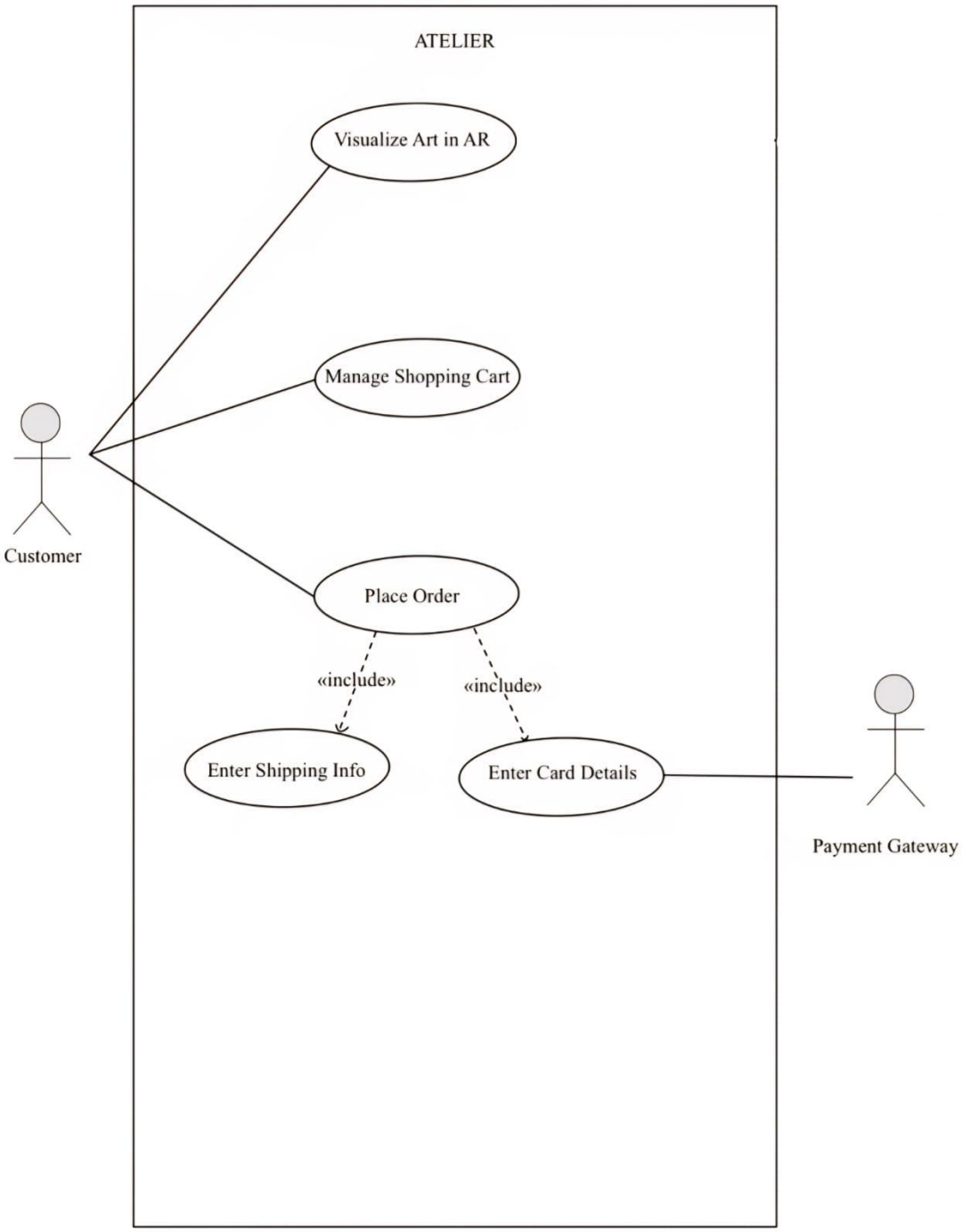
1.3 Non-Functional Requirements:

NFR-2: The website should be useable on both mobile phones and desktop computers

NFR-4: The system shall maintain full functionality on Chrome, Firefox, Safari, and Edge.

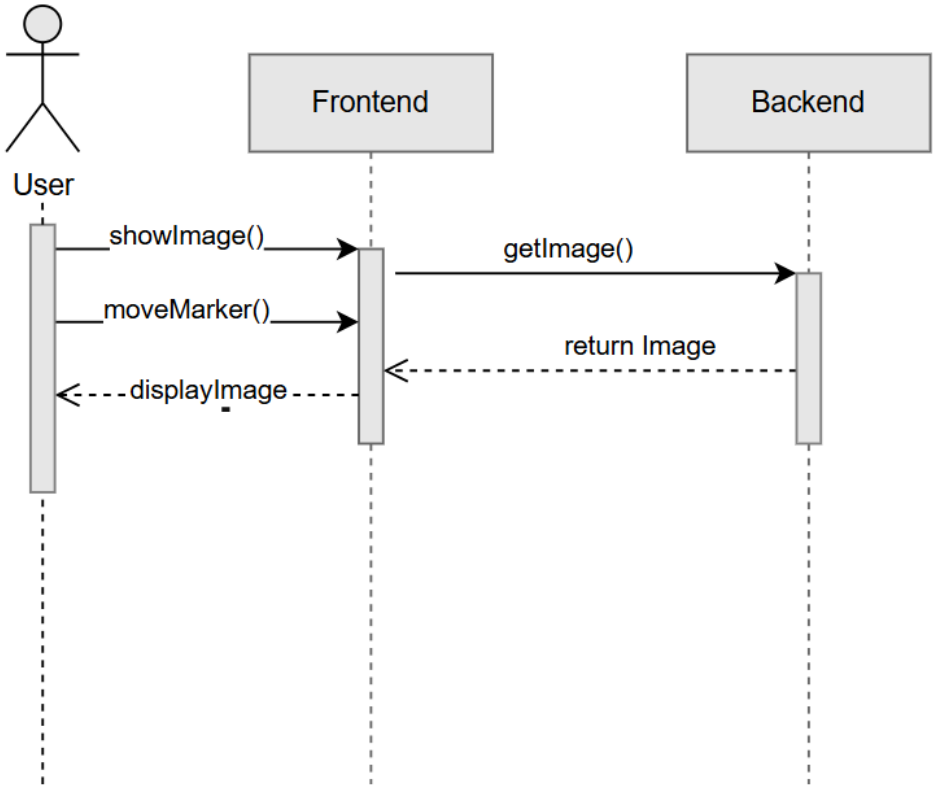
2 Diagrams

2.1 Use case:

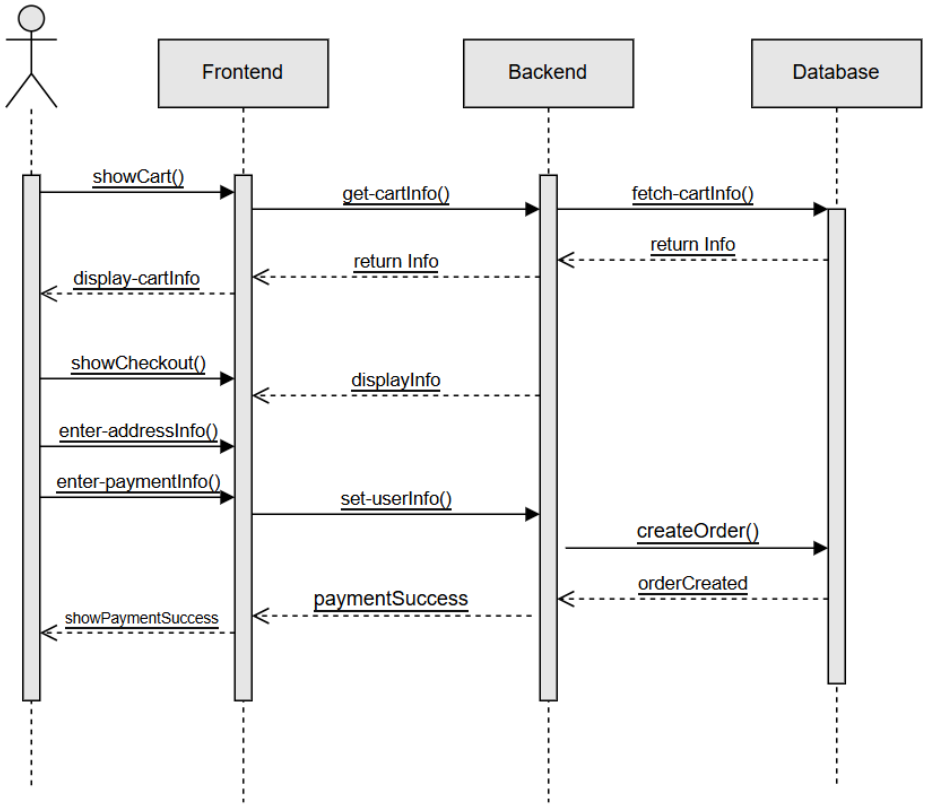


2.2 Sequence Diagram:

2.1 AR image display:



2.2 place order:



3 Stories:

3.1 Visualize Art in AR

Component Name: Web-AR

Story Name: Visualize Art in AR

Story Sequence No: 01

Story Short Description: Users can visualize paintings in their environment using AR.

Story Long Description: The user clicks the AR icon on a painting product page to open a new tab that activates the device camera, allowing them to visualize the painting in their environment using marker-based AR.

3.2 Manage Shopping Cart

Component Name: Cart

Story Name: Manage Shopping Cart

Story Sequence No: 02

Story Short Description: Users can add, view, and remove products from their shopping cart.

Story Long Description: The user adds products by clicking the cart icon, views all items grouped by artisan in the cart page with calculated totals, and removes products by clicking the cart icon again.

3.3 Place Order

Component Name: Checkout

Story Name: Place Order

Story Sequence No: 02

Story Short Description: Users can place an order for items in their cart.

Story Long Description: The user navigates checkout from the cart page, enters shipping information and card details, and completes the order after validation.

4 Meetings:

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*2*
Progress Check In Meeting
[18 November 2025]

Sprint Duration: ...
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:
Components Name:
- Cart
- Checkout
- Web-AR

Follow-up meeting questions:
13. What has been completed since the last meeting?
14. What are you going to be working on next?
15. Do you have any issues/impediments?

Scrum's Master comments based on the above questions:
13- Tasks are distributed among the team and we started researching web AR
14- Complete the front end and back end for the cart and payment pages
15- Finding AR methods that are completable with different devices

Sprint Meeting(s)

Project Name: Atelier
Project Members:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

---

Sprint #*2*
Progress Check In Meeting
[3 December 2025]

Sprint Duration: ...
Scrum Master: Shumokh Alsharif
Client: Malak Alharbi
Pair Programmers:
Remas Alsulami - Marwa Hadaddi - Reman Alamoudi - Ghaida Almehmadi - Shumokh Alsharif

Stories:

Components Name:

- Cart
- Checkout
- Web-AR

Follow-up meeting questions:

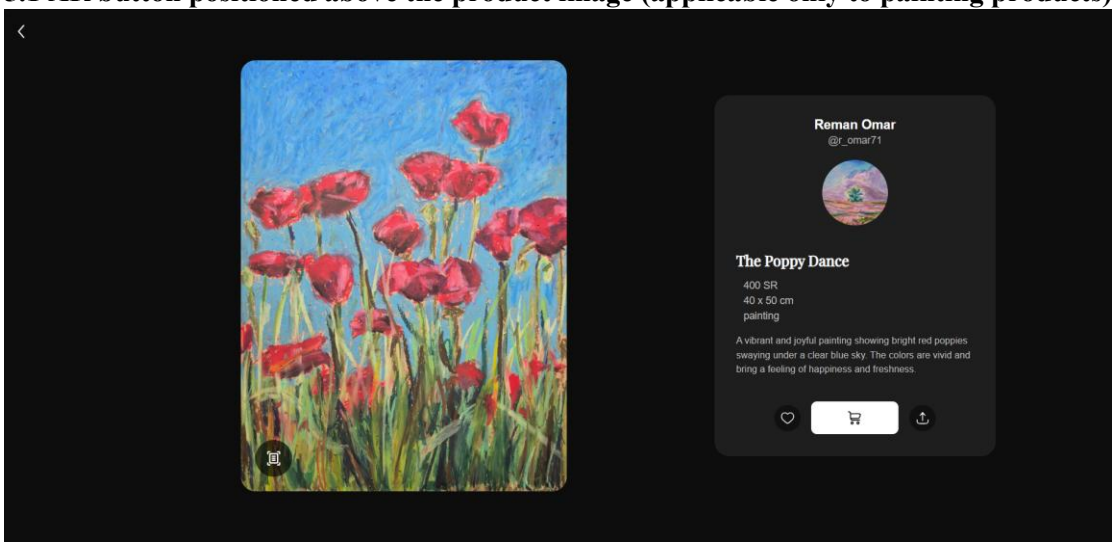
16. What has been completed since the last meeting?
17. What are you going to be working on next?
18. Do you have any issues/impediments?

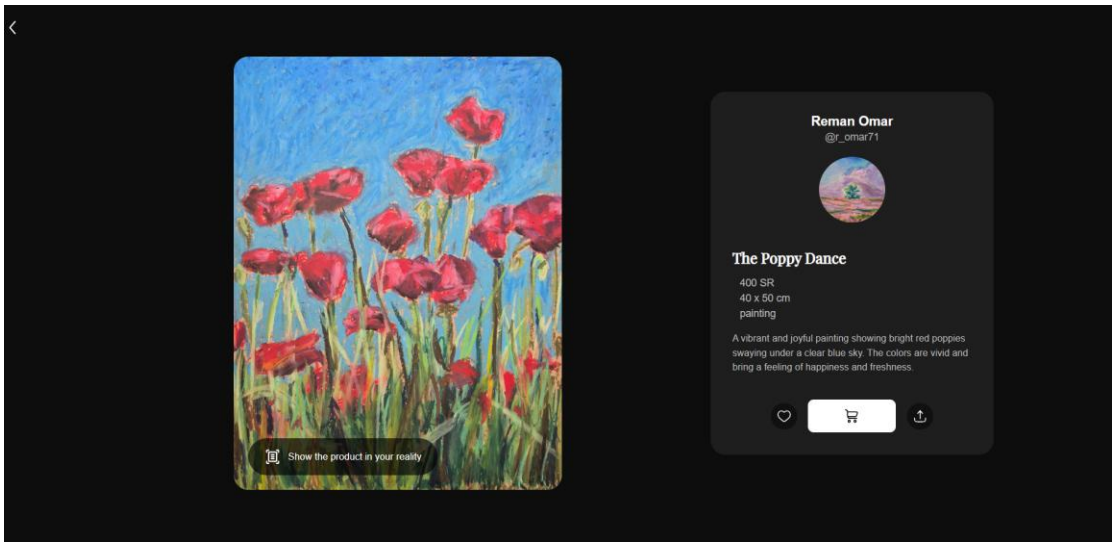
Scrum's Master comments based on the above questions:

16- All new endpoints are added to the server, the frontend pages and complete, the web AR is fully working
17- Refining and integrating what has been built so far to work together seamlessly
18- None

5 Implementation in Web browser (UI):

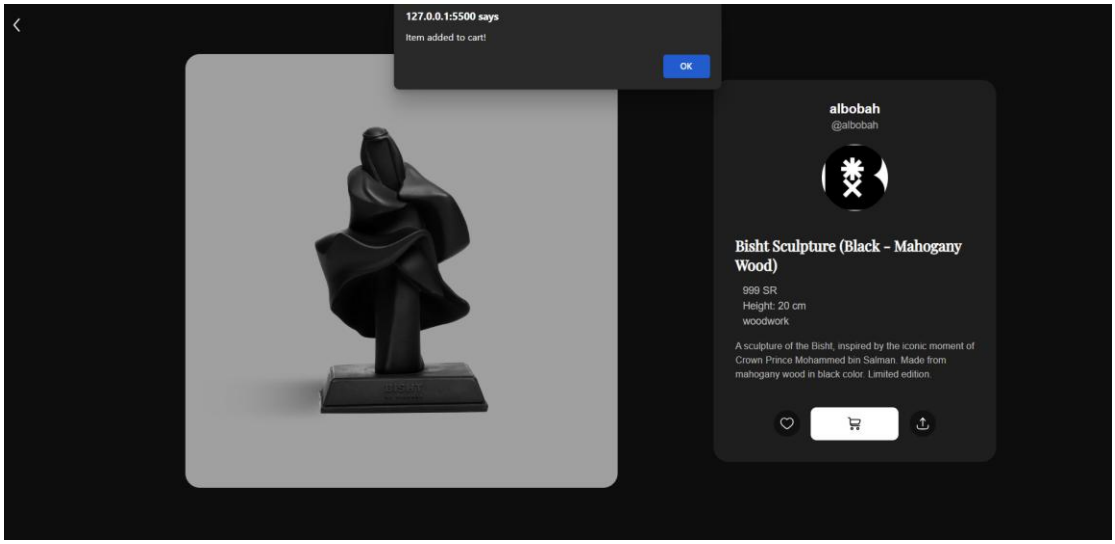
5.1 AR button positioned above the product image (applicable only to painting products)



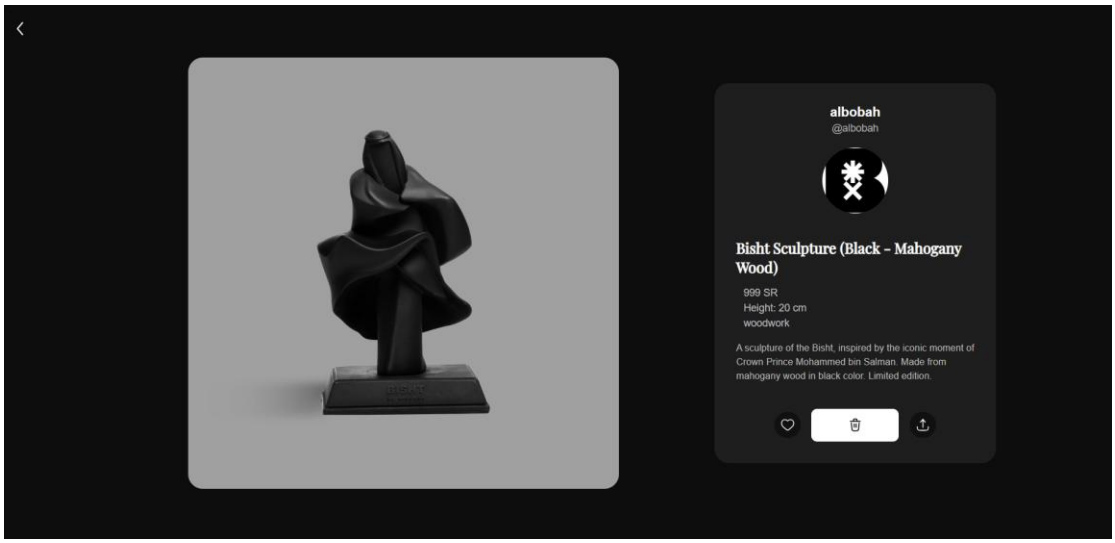


5.2 AR Feature

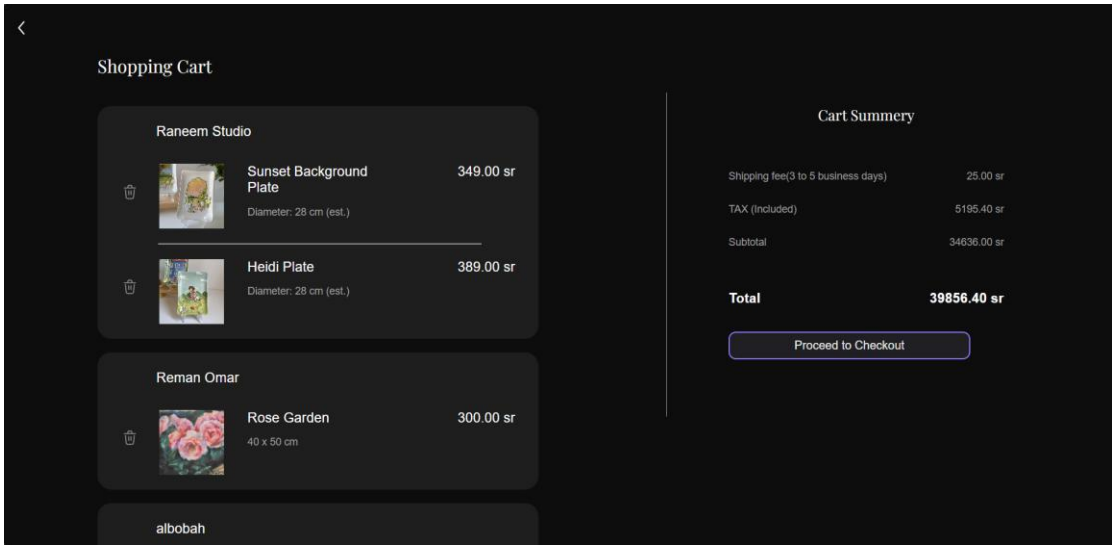
5.3 Add to cart button on product page



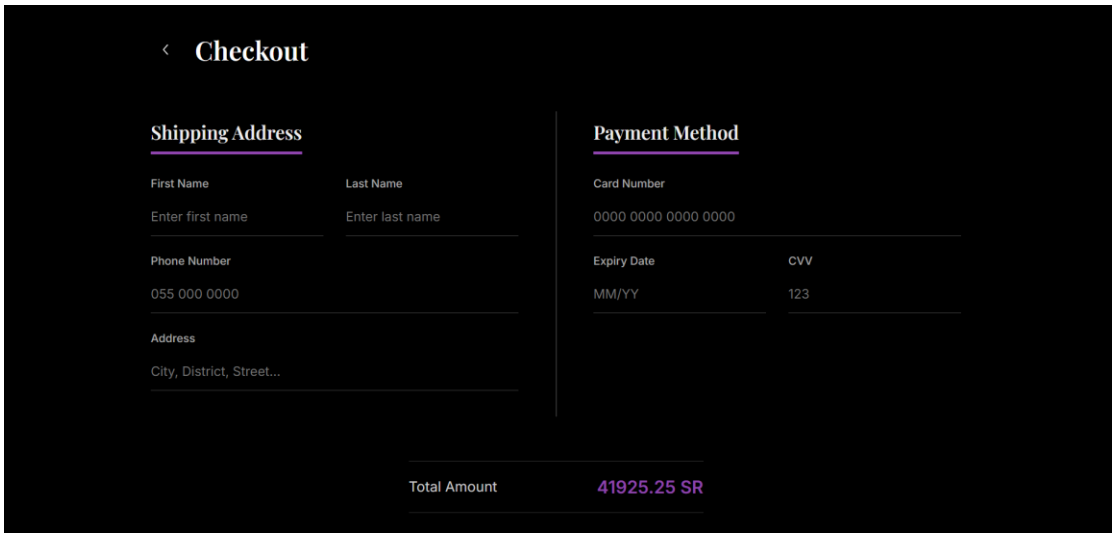
5.4 Delete from the cart button on product page

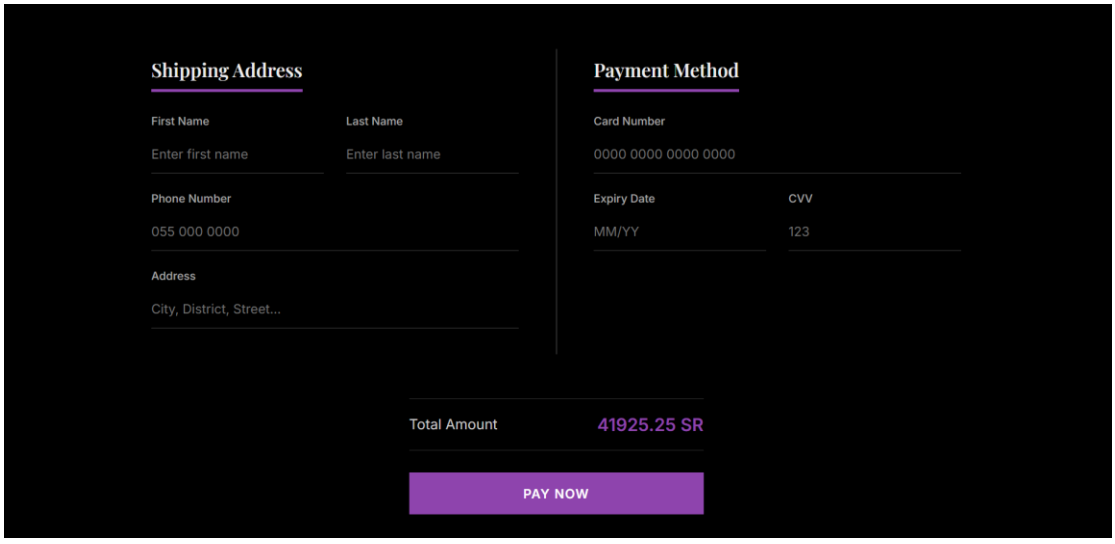


5.5 Cart page



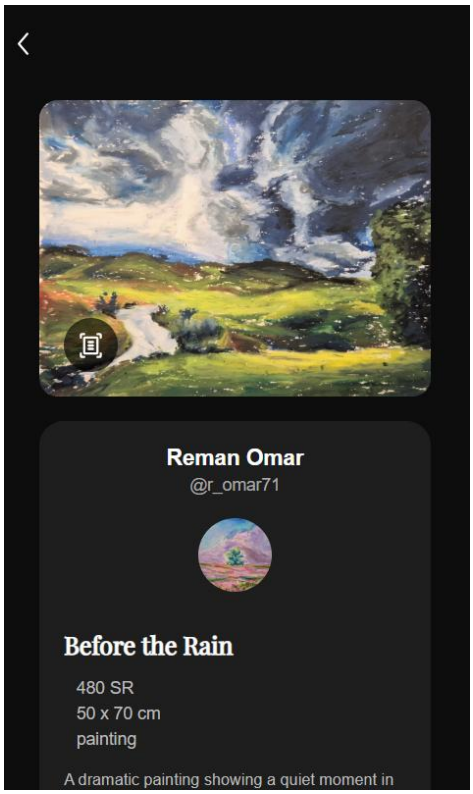
5.6 Checkout page

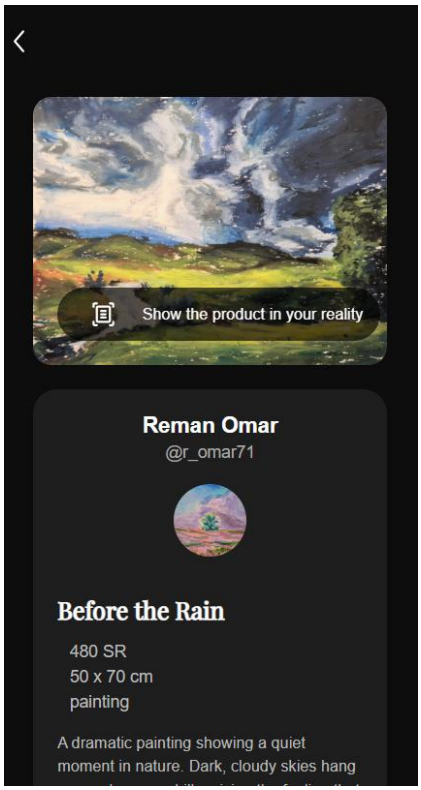




6 Implementation in Mobile (UI):

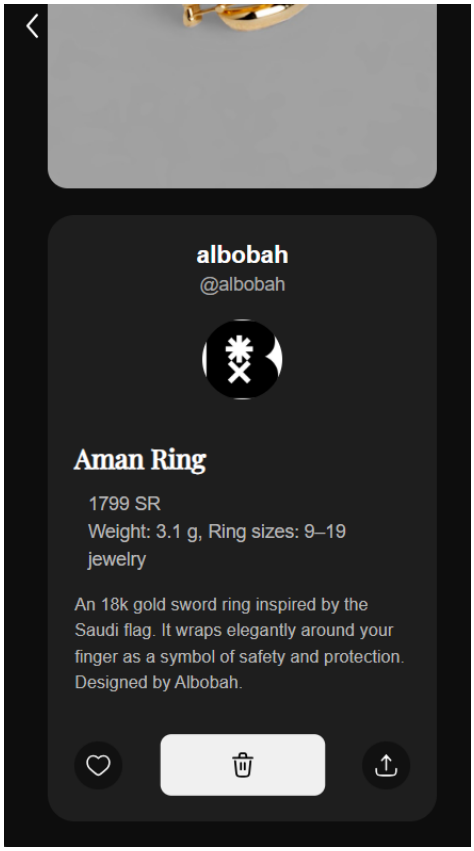
6.1 AR button positioned above the product image (applicable only to painting products)



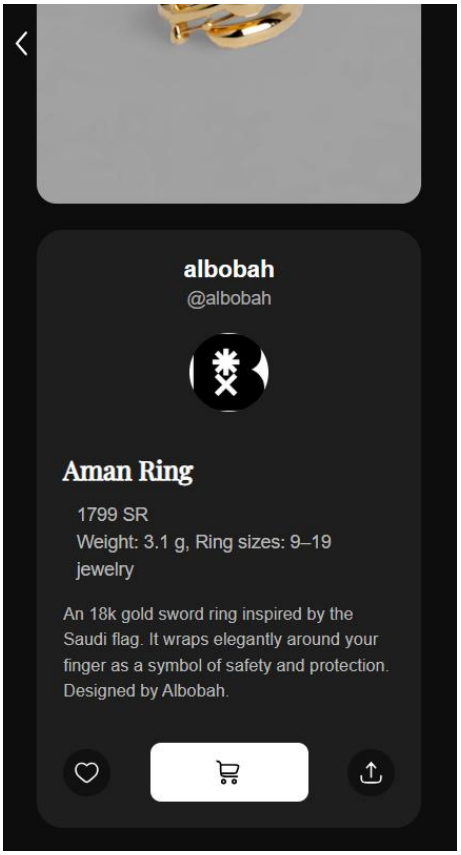


6.2 AR Feature

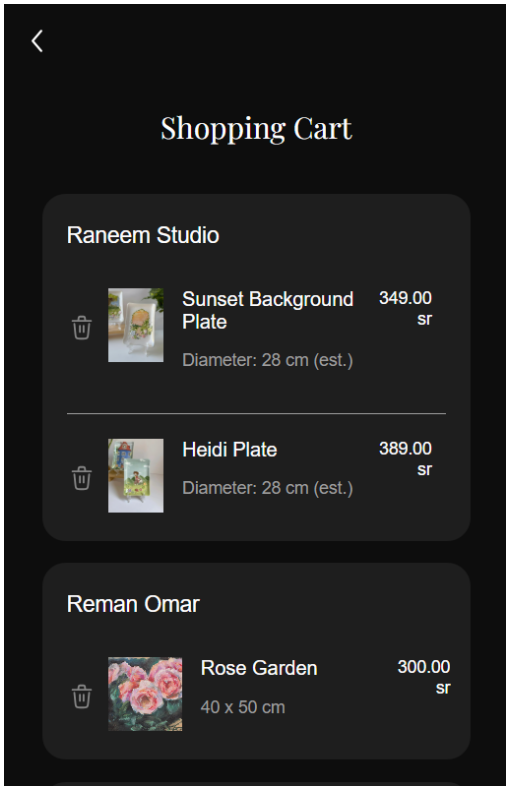
6.4 Delete from the cart button on product page

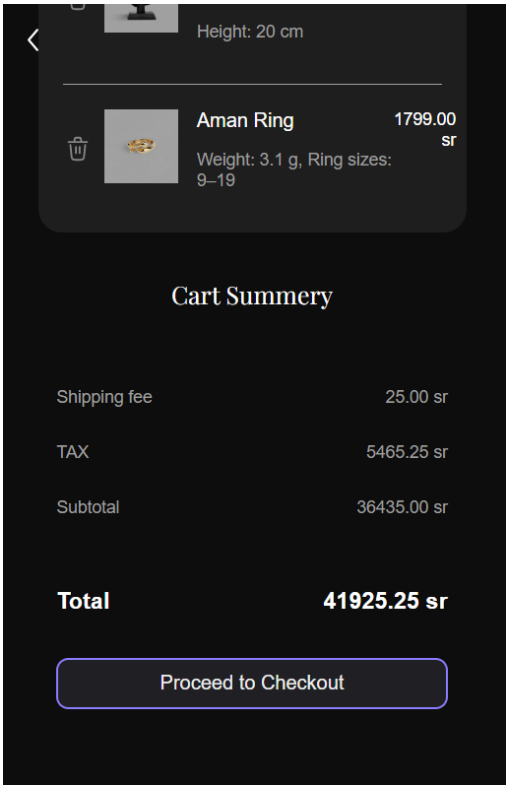


6.3 Add to cart button on product page

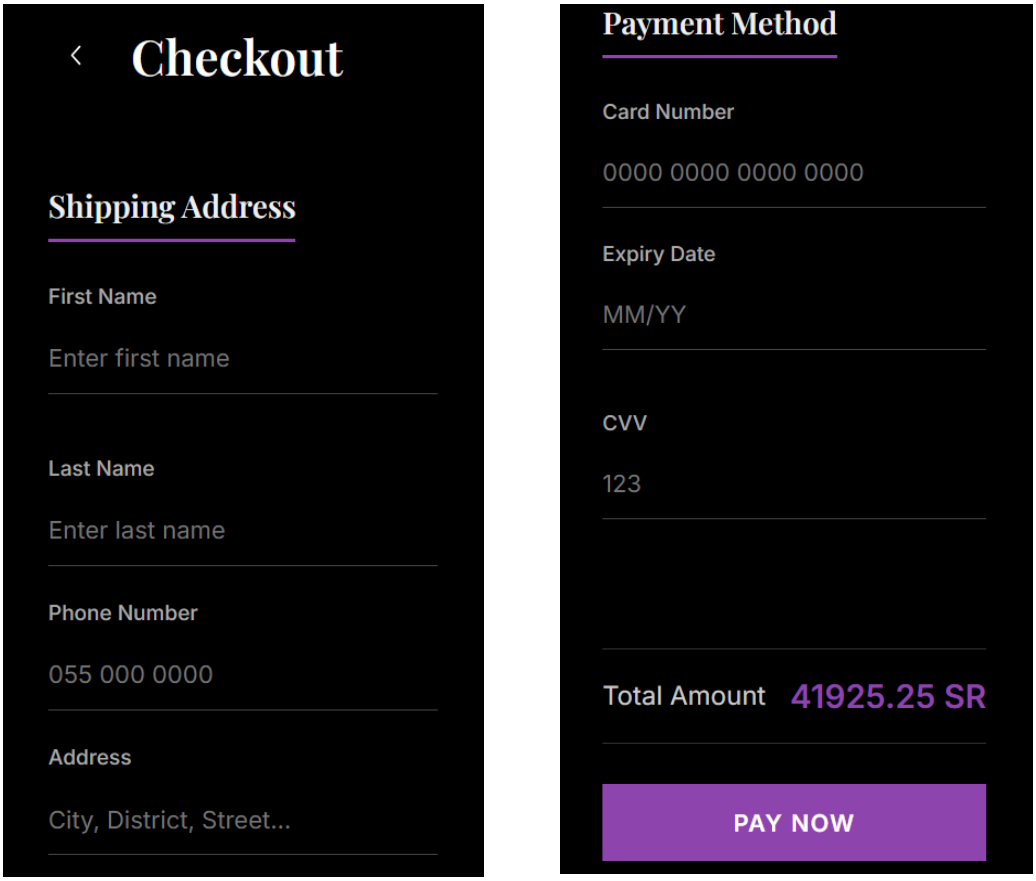


6.5 Cart page


Shopping Cart
Raneem Studio
Sunset Background Plate
349.00 sr
Diameter: 28 cm (est.)
Heidi Plate
389.00 sr
Diameter: 28 cm (est.)
Reman Omar
Rose Garden
300.00 sr
40 x 50 cm


Height: 20 cm
Aman Ring
1799.00 sr
Weight: 3.1 g, Ring sizes: 9–19
Cart Summery
Shipping fee
25.00 sr
TAX
5465.25 sr
Subtotal
36435.00 sr
Total
41925.25 sr
Proceed to Checkout

6.6 Checkout page



7 Test Cases

Test Case Name: Sprint 3- Cart

Test Case Id: Atelier - Cart

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	Unregistered users try to add an item to the Cart	Product/collection won't be added and user will be prompted to log in first	Pass
TC002	Registered user clicks on the cart icon in product page	The product will be added to their cart	Pass
TC003	User clicks on the cart icon in home page	Transports user to Cart page	Pass
TC004	Registered user opens the cart page	All the products on their cart will be displayed	Pass
TC005	Unregistered user opens the cart page	Nothing is displayed and user's cart will be empty	Pass
TC006	Registered user clicks on the cart icon for an item already in the cart	Product will be removed from the cart	Pass
TC007	User change window size	The layout adjusts accordingly	Pass
TC008	User clicks on any product image on the cart's page.	User is navigated to the product's specific page (Product page).	Pass
TC009	User clicks the "Back" button from the cart page.	User successfully returns to the home page.	Pass
TC010	User clicks the "Back" button from the product page.	User successfully returns to the cart Page.	Pass
TC011	User clicks on the checkout button	User goes to the checkout page	Pass

Test Case Name: Sprint 3- Checkout

Test Case Id: Atelier – Checkout

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User forgets a required field	The required field will vibrate, and the user won't continue until it is filled	Pass
TC002	User places their order	A confirmation message is displayed on screen and cart is emptied	Pass
TC003	User change window size	The layout adjusts accordingly	Pass
TC004	User clicks the "Back" button from the checkout page.	User successfully returns to the cart page.	Pass

Test Case Name: Sprint 3- Web-AR

Test Case Id: Atelier – Web-AR

Test Case No.	Test Case Description	Expected Results	Outcome (Pass/Fail) Other (Comments))
TC001	User opens product a page (painting)	Web-AR icon is found over the image on the bottom left	Pass
TC002	User hovers over the Web-Air icon	Icon expands over hovering	Pass
TC003	User clicks on Web-Air icon	A new tab opens for Web-AR	Pass
TC004	The camera detects the marker	The chosen painting is displayed with AR	Pass
TC005	User opens a product page (not paining)	The AR button is disabled	Pass
TC006	No marker found	The Web-AR will not display anything	Pass

Code:

1 Backend:

1.1 server:

server.js > app.post("/commission") callback > attachments

```
const mongoose = require("mongoose");
const express = require("express")
const bcrypt = require('bcrypt');
const cors = require('cors');
const jwt = require('jsonwebtoken');

const product = require("./models/product");
const artist = require("./models/artist");
const User = require('./models/user');
const Commission = require('./models/commission');

const JWT_SECRET = 'AA.201424';
const auth = require('./middleware/auth');


mongoose
  .connect("mongodb+srv://ghaida:GS.201424@cluster.cakgapc.mongodb.net/?retryWrites=true&w=majority&appName=Cluster")
  .then(() => {
    console.log(" connected successfully");
  })
  .catch((error) => {
    console.log(" error with connecting to the DB", error);
  });



const app = express()

app.use(express.json())
app.use(cors())
```

```
app.get("/", (req,res)=>{
  res.send("Welcom to the Atelier server root!!!")
})


app.get("/products", async (req, res) =>{

  try{
    const products = await product.find()
    console.log("Fething products completed!" )
    res.json(products)
    return
  }catch (error) {
    console.log("error while fetching products ", error)
    return res.send("error ")
  }


})


app.get("/products/category/:category", async (req, res) =>{

  try{
    const  catagory  = req.params.category

    const productsByCategory = await product.find({ category: catagory }).lean();

    if(!productsByCategory){
      return res.send("No product was found for this category")
    }
```

```
    // const collections = await product.distinct("collections", { catagory })
    // const collections = [...new Set(productsByCategory.map(p => p.collections || p.collections?.[0]))]
    const collections = [
  ...new Set(
    productsByCategory.map(p => Array.isArray(p.collections) ? p.collections[0] : p.collections)
  )
];


    // res.json(productsByCategory)

    res.json({
      collections: collections,
      products: productsByCategory
    });
    console.log("Fething products and collections by category is completed!" )
    return

  }catch (error) {
    console.log("error while fetching products by category", error)
    return res.send("error")
  }


})


```

```
app.get("/products/id/:productId", async (req, res) =>{
    const id = req.params.productId

  try{

    const productById = await product.findById(id)
    if(!productById){
      return res.send("No product was found with this id")
    }

    // res.json(productById)

    const artistById = await artist.findOne({ artistId: productById.artistId  })
    if(!artistById){
      return res.send("No artist was found with this id")
    }

    res.json({
      product: productById,
      artist: artistById
    })
    console.log("Fething products by ID is completed!" )
    return

  }catch (error) {
    console.log("error while fetching products by ID")
    return res.send("error")
  }


})

```

```
app.get("/products/collection/:collection", async (req, res) =>{

  try{
    const collection = req.params.collection

    const collectionProducts= await product.find({ collections: collection})
    if(!collectionProducts){
      return res.send("No product was found in this collection")
    }

    // res.json(collectionProducts)
    const artistById = await artist.findOne({ artistId: collectionProducts[0].artistId });
    if (!artistById) {
      return res.send("No artist was found for this collection");
    }

    res.json({
      products: collectionProducts,
      artist: artistById
    });

    console.log("Fething products in the collection is completed!" )
    return

  }catch (error) {
    console.log("error while fetching the collection")
    return res.send("error")
  }

})
```

```
app.get("/collections", async (req, res) => {
  try {
    const collections = await product.distinct("collections");
    if (!collections || collections.length === 0) {
      return res.send("No collections found");
    }

    res.json(collections);
    console.log("Fetching all unique collections completed!");
  } catch (error) {
    console.log("Error while fetching collections:", error);
    res.send("error");
  }
});


app.get("/collections/artist/:artistId", async (req, res) => {
  try {

    const artistId = req.params.artistId;
    const collections = await product.distinct("collections", { artistId: artistId });


    if (!collections || collections.length === 0) {

      return res.send("No collections found for this artist");
    }

    //fetch product images
    const results = [];

    for (const collection of collections) {
      const productsInCollection = await product
```

```
    for (const collection of collections) {
      const productsInCollection = await product
        .find({ artistId, collections: collection })
        .limit(2)
        .select("images");


      const images = productsInCollection.map(p => p.images[0]).filter(Boolean);

      results.push({
        collection,
        images
      });
    }

    res.json(results);
    console.log("Fetching collections by artist completed!");
  } catch (error) {
    console.log("Error while fetching collections by artist: ", error);
    res.send("error");
  }

});


app.get("/products/artistId/:artistId", async (req, res) =>{

  try{
    const  artistId  = req.params.artistId

    const productsByArtisan = await product.find({ artistId: artistId })
```

```
app.get("/products/artistId/:artistId", async (req, res) =>{

  try{
    const  artistId  = req.params.artistId

    const productsByArtisan = await product.find({ artistId: artistId })

    if (!productsByArtisan || productsByArtisan.length === 0) {
      return res.send("No product was found for this artist")
    }


    const artistInfo = await artist.findOne({ artistId })

    if (!artistInfo) {
      return res.send("No artist info found for this artist")
    }

    res.json({
      artist: artistInfo,
      products: productsByArtisan
    })



  }catch (error) {
    console.log("error while fetching products by artisan", error)
    return res.send("error")
  }


})

```

```
247
248 app.get("/products/artistId/available/:artistId", async (req, res) => {
249   try {
250     const artistId = req.params.artistId;
251
252     const inStockProducts = await product.find({
253       artistId: artistId,
254       availability: "In Stock"
255     });
256
257     res.json(inStockProducts);
258
259   } catch (error) {
260     console.log("Error while fetching in-stock products by artist:", error);
261     return res.send("Error fetching in-stock products");
262   }
263 });
264
265
266 // Wishlist Endpoints
267 const { Types: { ObjectId } } = mongoose;
268
269 function isValidObjectId(id) {
270   return ObjectId.isValid(id);
271 }
272
273 /**
274  * retrieves the full wishlist
275  */
276 app.get("/users/:userId/wishlist", async (req, res) => {
277   try {
278     const { userId } = req.params;
279     if (!isValidObjectId(userId)) {
```

```
278     const { userId } = req.params;
279     if (!isValidObjectId(userId)) {
280       return res.status(400).json({ message: "Invalid userId" });
281     }
282
283     const user = await User.findById(userId, { wishList: 1, _id: 0 }).lean();
284     if (!user) return res.status(404).json({ message: "User not found" });
285
286     return res.status(200).json(Array.isArray(user.wishList) ? user.wishList : []);
287   } catch (err) {
288     console.error("GET /wishlist error:", err);
289     return res.status(500).json({ message: "Error fetching wishlist" });
290   }
291 });
292
293 /**
294  * retrieves only the productIds from the wishlist
295  */
296 app.get("/users/:userId/wishlist/products", async (req, res) => {
297   try {
298     const { userId } = req.params;
299     if (!isValidObjectId(userId)) {
300       return res.status(400).json({ message: "Invalid userId" });
301     }
302
303     const user = await User.findById(userId, { wishList: 1, _id: 0 }).lean();
304     if (!user) return res.status(404).json({ message: "User not found" });
305
306     const productIds = (user.wishList || [])
307       .filter(item => item && item.product)
308       .map(item => item.product);
309
```

```
    return res.status(200).json(productIds);
  } catch (err) {
    console.error("GET /wishlist/products error:", err);
    return res.status(500).json({ message: "Error fetching product wishlist" });
  }
});

/**
 * adds/removes a product from the wishlist
 */
app.put("/users/:userId/wishlist/products/toggle", async (req, res) => {
  try {
    const { userId }   = req.params;
    const { productId, collection } = req.body;

    if (!isValidObjectId(userId) || !isValidObjectId(productId)) {
      return res.status(400).json({ message: "Invalid userId or productId" });
    }

    // to verify product exists
    const exists = await product.exists({ _id: productId });
    if (!exists) return res.status(404).json({ message: "Product not found" });

    const user = await User.findById(userId);
    if (!user) return res.status(404).json({ message: "User not found" });

    const list = user.wishList || [];
    const index = list.findIndex(
      item => item.product && item.product.toString() === productId
    );

    let toggled;
```

```
    let toggled;

    if (index > -1) {
      // remove
      list.splice(index, 1);
      toggled = "removed";
    } else {
      // add
      list.push({
        product: productId,
        collection: collection || null,
        dateAdded: new Date()
      });
      toggled = "added";
    }

    user.wishList = list;
    await user.save();

    return res.status(200).json({
      toggled,
      wishlist: user.wishList
    });
  } catch (err) {
    console.error("PUT /wishlist/products/toggle error:", err);
    return res.status(500).json({ message: "Error toggling product wishlist" });
  }
});

/**
 * retrieves wishlist collections with counts
 */
```

```
402  /**
403   * adds/removes a collection bookmark from the wishlist
404   */
405  app.put("/users/:userId/wishlist/collections/toggle", async (req, res) => {
406    try {
407      const { userId }   = req.params;
408      const { collection } = req.body;
409
410      if (!isValidObjectId(userId)) {
411        return res.status(400).json({ message: "Invalid userId" });
412      }
413
414      if (!collection || typeof collection !== "string") {
415        return res.status(400).json({ message: "Collection is required" });
416      }
417
418      const user = await User.findById(userId);
419      if (!user) return res.status(404).json({ message: "User not found" });
420
421      const list = user.wishList || [];
422
423      // look for bookmark (product: null)
424      const index = list.findIndex(
425        (item) =>
426          item &&
427          (item.product === null || item.product === undefined) &&
428          item.collection === collection
429      );
430
431      let toggled;
432
```

```
433      if (index > -1) {
434        //if found → remove bookmark
435        list.splice(index, 1);
436        toggled = "removed";
437      } else {
438        //if not found → add bookmark
439        list.push({
440          product: null,
441          collection,
442          dateAdded: new Date(),
443        });
444        toggled = "added";
445      }
446
447      user.wishList = list;
448      await user.save();
449
450      return res.status(200).json({
451        toggled,
452        wishlist: user.wishList,
453      });
454    } catch (err) {
455      console.error("PUT /wishlist/collections/toggle error:", err);
456      return res
457        .status(500)
458        .json({ message: "Error toggling collection wishlist" });
459    }
460  });
461
```

```
// End Wishlist Endpoints

app.post("/product", async (req, res) => {
  try {
    const newProduct = new product(req.body);

    await newProduct.save();

    res.json(newProduct);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});


// Create new artist
app.post("/artists", async (req, res) => {
  try {
    const newArtist = new artist(req.body);
    await newArtist.save();
    res.json(newArtist);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
});


const { sendEmail } = require('./email');
const user = require("./models/user");

app.post("/commission", async (req, res) => {
  try {

```

```
    const {
      type,
      dimensions,
      attachment,
      description,
      phoneNumber,
      country,
      city,
      isGift,
      artistname,
      artistEmail,
      userEmail,
      username

    } = req.body;

    const newCommission ={
      type,
     dimensions,
     attachment,
     description,
     phoneNumber,
     country,
     city,
     isGift,
     artistname,
     artistEmail,
     userEmail,
     dateSubmitted: new Date()
    }

```

```
console.log('New commission requested: ', newCommission)


console.log("tesssst daaataa:", {
        artistname,
        description
    });


// artist email
    try {

      await sendEmail({
        to: "remasalsulami962@gmail.com",
        subject: "New Commission Request",
        template: 'artisanView',
        context: {
            artistname: artistname,
            username: username,
            details: description
          },
        attachments: [
          {
            filename: 'Email_icon.svg',
            path: __dirname + '/views/Group6.svg',
            cid: 'mail@atelier'
          }
        ]
    })} catch (error) {
      console.error("Email error: ", error);
    }
```

```
//  user email
  try {
    await sendEmail({
      to: username ,
      subject: "Your Commission Has Been Sent",
      template: 'userView',
      context: {
        username: username,
        artistname: artistname
      },
      attachments: [
      {
        filename: 'Email_icon.svg',
        path: __dirname + '/views/Group6.svg',
        cid: 'mail@atelier'
      }
    ]
  })} catch (error) {
    console.error("Email error:" , error);
  }


/*
  try {
    await sendEmail({
    to: userEmail,
    subject: "Your Commission Has Been Sent",
    template: 'userView',
    context: { username },
    attachments: [
```

```
    res.status(201).json({
      message: "Commission request received successfully! ",
      requestData: newCommission
    })

    // afte the comission schema is done
    // const savedRequest = await CommissionRequest.create(newCommission);
    // await sendCommissionEmail(newCommission.artistEmail, newCommission);
    // res.status(201).json({
    //   message: "Commission request received successfully!",
    //   requestData: savedRequest
    // });


  } catch (error) {
    console.error("Error while submitting commission request: ", error)
    res.status(500).json({ message: "Error while submitting commission" })
  }
})


app.post('/api/register', async (req, res) => {
  const { username, email, password } = req.body;

  try {
    const existingUser = await User.findOne({ $or: [{ email }, { username }] });

    if (existingUser) {
      const field = existingUser.email === email ? 'Email' : 'Username';
      return res.status(400).json({ message: `${field} is already in use.` });
    }
```

```
    const newUser = new User({ username, email, password });
    await newUser.save();

    res.status(201).json({
      message: 'Registration successful!',
      user: {
        id: newUser._id,
        username: newUser.username,
        email: newUser.email
      }
    });

  } catch (error) {
    console.error('Registration Error:', error.message);
    res.status(500).json({ message: 'Server error during registration.' });
  }
});

app.post('/api/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ $or: [{ email }, { username: email }] });

    if (!user) {
      return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(401).json({ message: 'Authentication failed. Invalid username or password.' });
```

```
    const token = jwt.sign(
      { userId: user._id, email: user.email },
      JWT_SECRET,
      { expiresIn: '1d' }
    );

    res.status(200).json({
      message: 'Login successful!',
      token,
      user: {
        id: user._id,
        username: user.username,
        email: user.email
      }
    });

  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Server error during login.' });
  }

});


app.get("/users", async (req, res) =>{

  try{
    const users = await user.find()
    console.log("Fething users completed!" )
    res.json(users)
    return
```

```
  }catch (error) {
    console.log("error while fetching users ", error)
    return res.send("error ")
  }


})

app.get("/user/:userId", async (req,res) =>{
  const userId = req.params.userId
  try{
    const userinfo = await user.findById(userId).select("username email")

    if (!userinfo){
      return res.status(404).send("User not found");
    }

    res.send(userinfo)
    console.log("User info fetched successfully")

  }catch(error){
    console.log("error while fetching a user info ", error)
    res.status(500).send("Error while fetching user info")
  }
})

app.post("/cart/add", async (req, res) => {
  const { userId, productId, quantity } = req.body;

  if (!userId || !productId) {
    return res.status(400).json({ message: "Missing required fields: userId and productId are required" });
  }
```

```
  }

  try {
    const user = await User.findById(userId);

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    const qty = quantity && quantity > 0 ? quantity : 1;

    const itemIndex = user.cart.findIndex(item => item.product.toString() === productId);

    if (itemIndex > -1) {
      user.cart[itemIndex].quantity += qty;
    } else {
      user.cart.push({
        product: productId,
        quantity: qty,
        dateAdded: new Date()
      });
    }

    await user.save();

    res.status(200).json({
      message: "Cart updated successfully",
      cart: user.cart
    });

  } catch (error) {
    console.error("Error in /cart/add:", error);
```

```
    res.status(500).json({ message: "Error adding to cart" });
  }
});

app.delete("/cart/remove", async (req, res) => {
  const { userId, productId } = req.body;

  if (!userId || !productId) {
    return res.status(400).json({ message: "Missing userId or productId" });
  }

  try {
    const user = await User.findById(userId);

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    const originalLength = user.cart.length;

    user.cart = user.cart.filter(item => item.product.toString() !== productId);

    if (user.cart.length === originalLength) {
      return res.status(404).json({ message: "Product not found in cart" });
    }

    await user.save();

    res.status(200).json({
      message: "Item removed successfully",
      cart: user.cart
    });
```

```
799
800   } catch (error) {
801     console.error("Error in /cart/remove:", error);
802     res.status(500).json({ message: "Error removing item from cart" });
803   }
804 });
805
806 app.get('/cart/:userId', async (req, res) => {
807     try {
808         const { userId } = req.params;
809
810         const user = await User.findById(userId).populate({
811             path: 'cart.product',
812             model: 'product',
813             select: 'name images price dimensions artistId'
814         });
815
816         if (!user) {
817             return res.status(404).json({ message: 'User not found' });
818         }
819
820         let subTotal = 0;
821         const groupsObj = {};
822
823         for (const cartItem of user.cart) {
824     if (!cartItem.product) continue;
825
826     const product = cartItem.product;
827
828     const lineTotal = product.price;
829     subTotal += lineTotal;
830
```

```
831     const itemData = {
832         productId: product._id,
833         name: product.name,
834         picture: product.images?.[0] || '',
835         price: product.price,
836         dimensions: product.dimensions,
837         lineTotal
838     };
839
840     //  Use findOne with custom artistId
841     const artistDoc = await artist.findOne({ artistId: String(product.artistId) });
842     const artistName = artistDoc?.name || "Unknown Artist";
843
844     if (!groupsObj[artistName]) {
845         groupsObj[artistName] = [];
846     }
847
848     groupsObj[artistName].push(itemData);
849 }
850
851
852         const groupedCart = Object.keys(groupsObj).map(artist => ({
853             artist: artist,
854             items: groupsObj[artist]
855         }));
856
857         const TAX_RATE = 0.15;
858         const taxAmount = subTotal * TAX_RATE;
859         const shipping = 25;
860         const total = shipping + subTotal + taxAmount;
861
862         res.status(200).json({
```

```
        if (!user) {
            return res.status(404).json({ message: "User not found" });
        }

        // Key Action: Clear the entire cart array
        user.cart = [];

        await user.save();

        res.status(200).json({
            message: "Cart cleared successfully.",
            cart: user.cart
        });

    } catch (error) {
        console.error("Error in /cart/clear:", error);
        res.status(500).json({ message: "Error clearing cart" });
    }
});


app.post("/checkout", async (req, res) => {
  const { userId } = req.body;

  if (!userId) {
    return res.status(400).json({ message: "UserId is required" });
  }

  try {
    const user = await User.findById(userId);

    if (!user) {
```

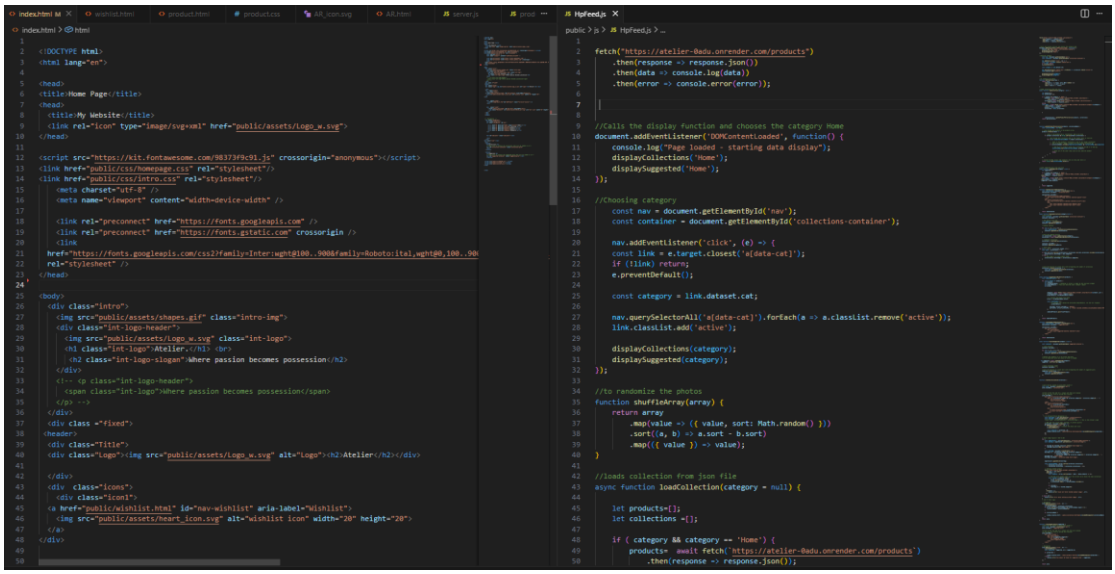
```
  const { userId } = req.body;

  if (!userId) {
    return res.status(400).json({ message: "UserId is required" });
  }

  try {
    const user = await User.findById(userId);

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    user.cart = [];

    await user.save();

    res.status(200).json({ message: "Checkout successful and cart cleared!" });
    console.log(`Cart cleared for user: ${userId}`);

  } catch (error) {
    console.error("Error during checkout:", error);
    res.status(500).json({ message: "Server error during checkout" });
  }
});


app.listen(3000, () =>{
    console.log("I'm listening in the port 3000")
})
```

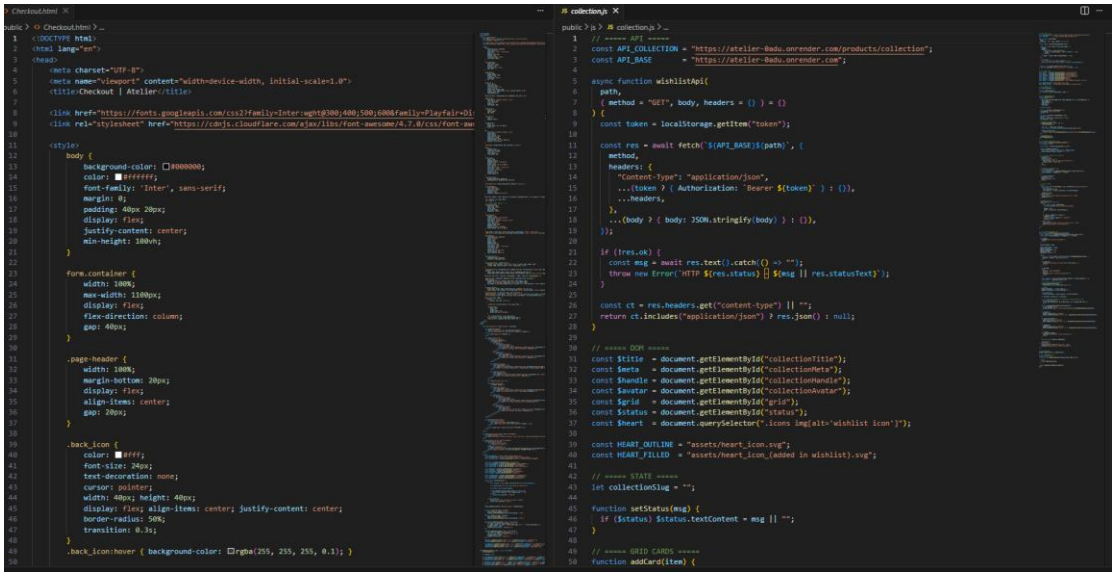
1.2 AR:

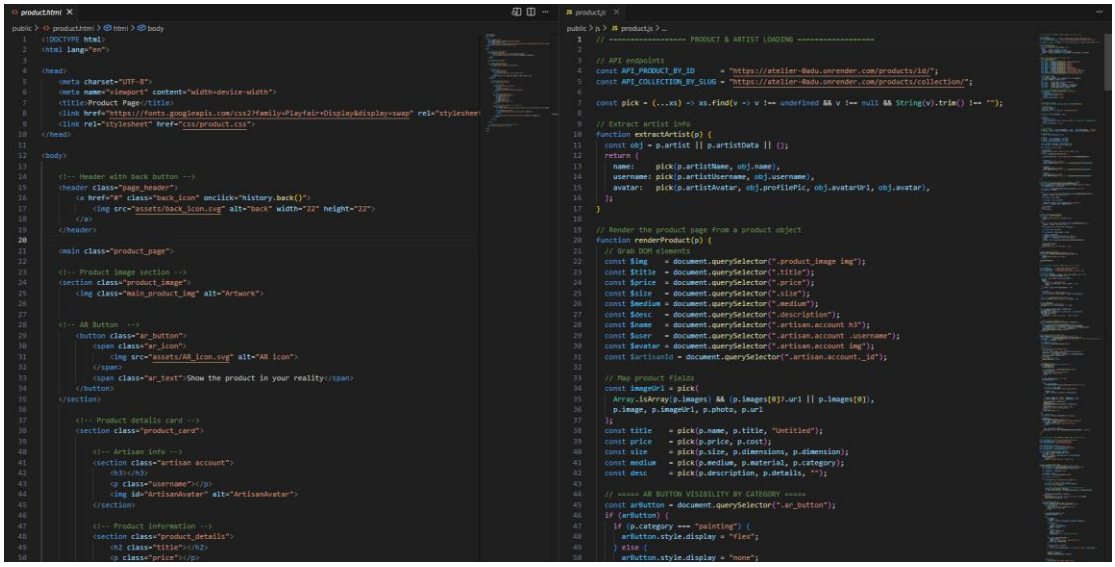
```html
<!doctype html>
<html>
  <head>

    <script src="https://aframe.io/releases/1.2.0/aframe.min.js"></script>
    <script src="https://cdn.jsdelivr.net/gh/AR-js-org/AR.js@3.3.2/aframe/build/aframe-ar.js"></script>
  </head>

  <body style="margin: 0; overflow: hidden;">
    <a-scene
      embedded
      vr-mode-ui="enabled: false"
      renderer="logarithmicDepthBuffer: true;"
      arjs="trackingMethod: best; sourceType: webcam; debugUIEnabled: true;">


      <a-marker preset="hiro">
        <!--<a-box position="0 0.5 0" color="red"></a-box>  -->

       <!--<a-marker type="pattern" url="assets/logo-marker.patt">   -->
        <a-plane
          id="ar-image"
          src="ar-test.jpg"
          height="1"
          width="1.5"
          position="0 0.5 0"
          rotation="-90 0 0"
        ></a-plane>
      </a-marker>

      <a-entity camera></a-entity>
    </a-scene>
```

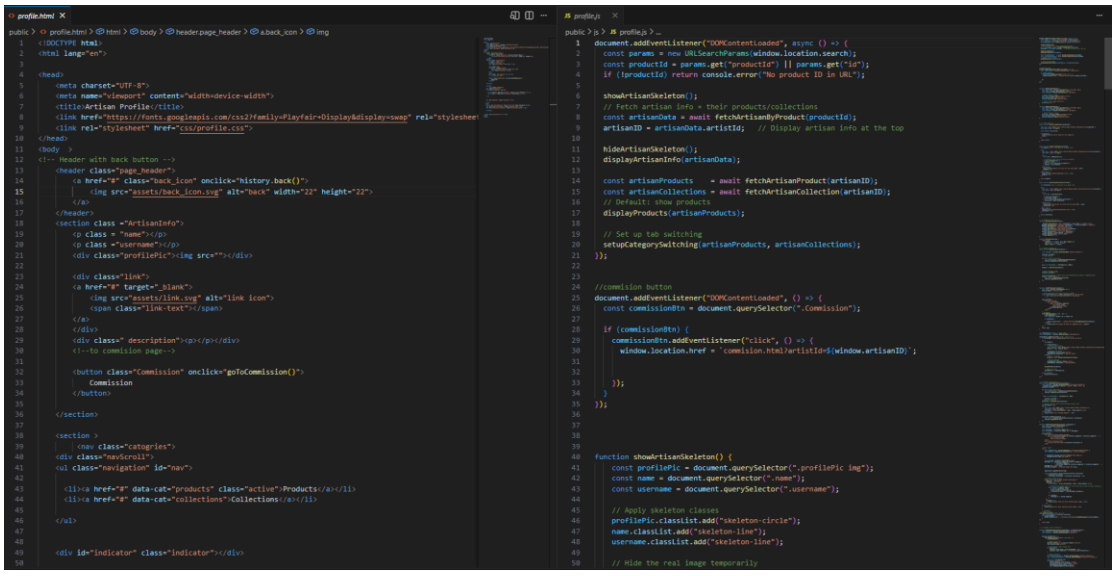
```html
    <script>

      function getGoogleDriveDirectUrl(url) {
        if (url.includes('drive.google.com')) {
          const match = url.match(/[?&]id=([^&]+)/);
          if (match) {
            const fileId = match[1];

            return `https://lh3.googleusercontent.com/d/${fileId}`;
          }
        }
        return url;
      }

      window.addEventListener('load', () => {
        const urlParams = new URLSearchParams(window.location.search);
        let imageUrl = urlParams.get('img');

        if (imageUrl) {
          imageUrl = getGoogleDriveDirectUrl(imageUrl);
          console.log('Loading AR image:', imageUrl);

          const arImage = document.getElementById('ar-image');
          arImage.setAttribute('src', imageUrl);
        }
      });

    </script>

  </body>
```

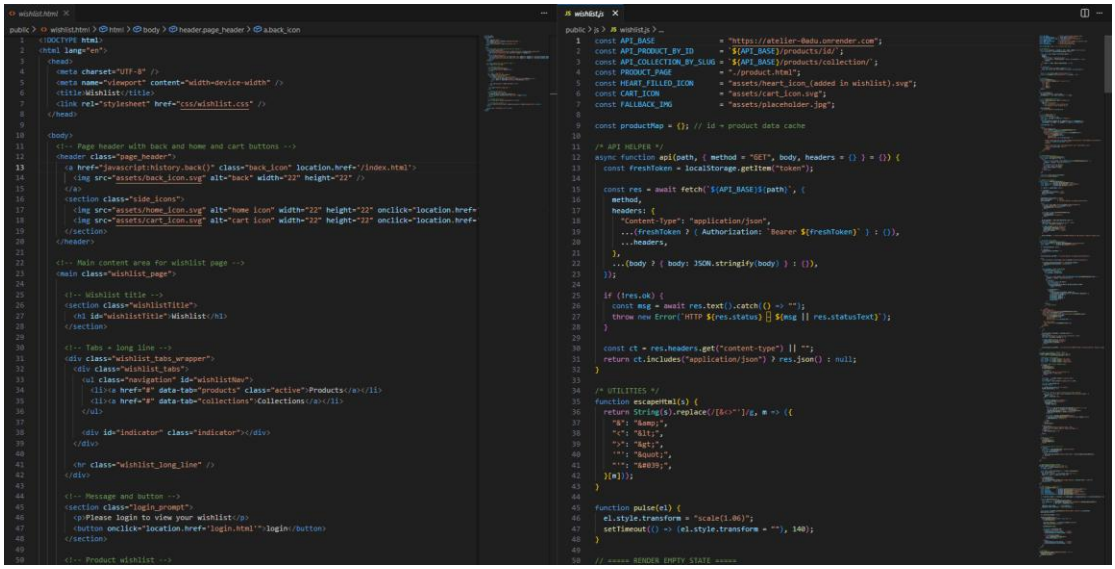
2 Frontend:

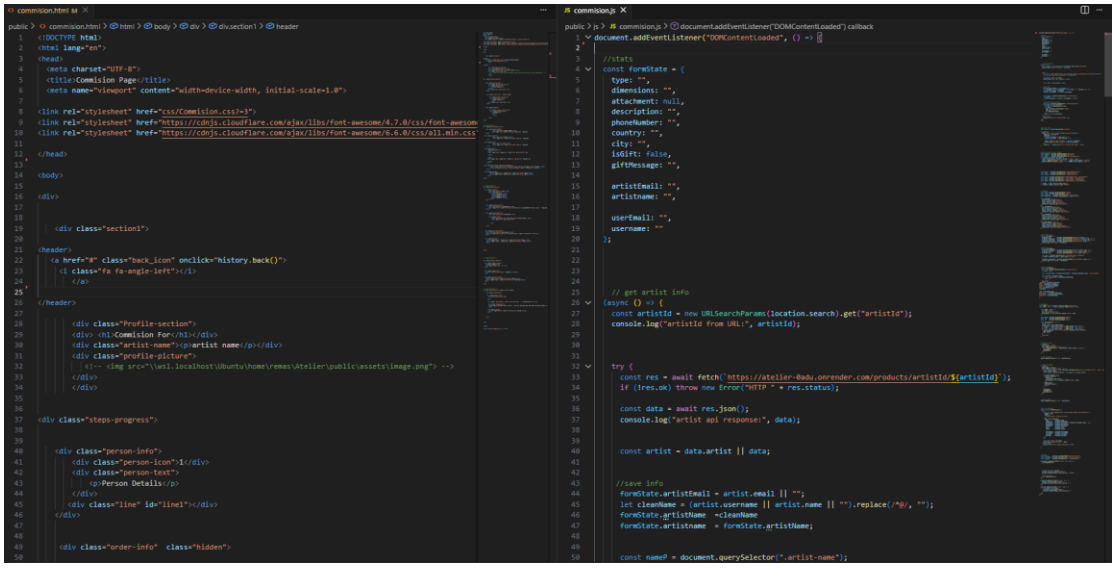


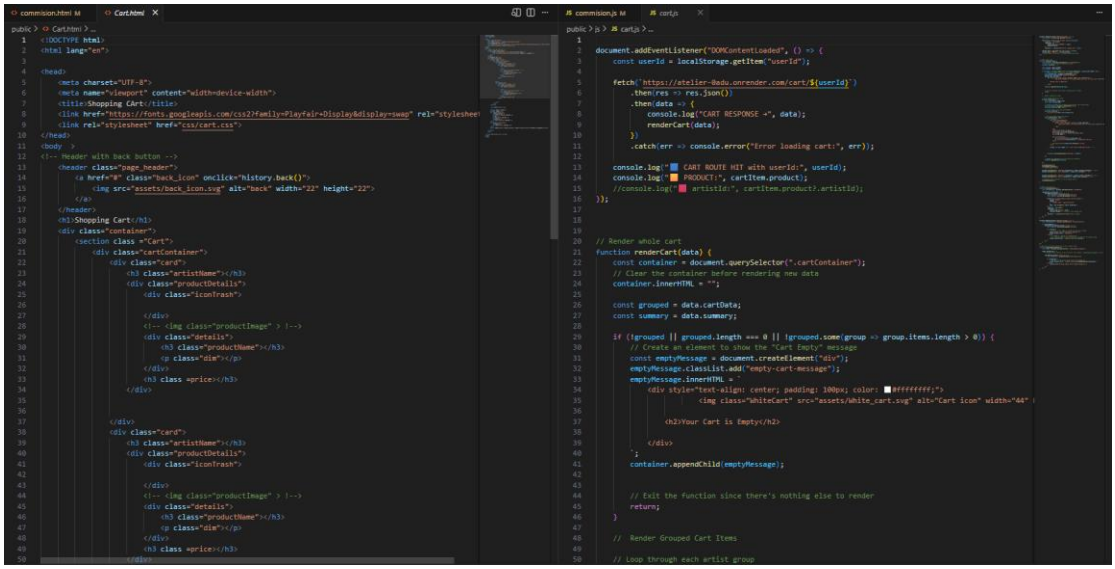


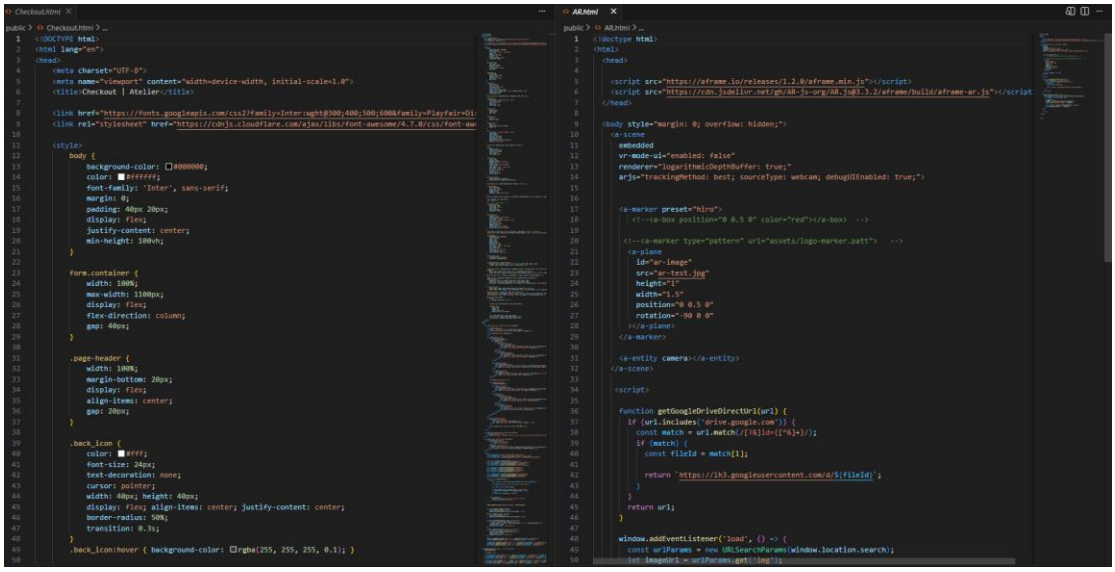


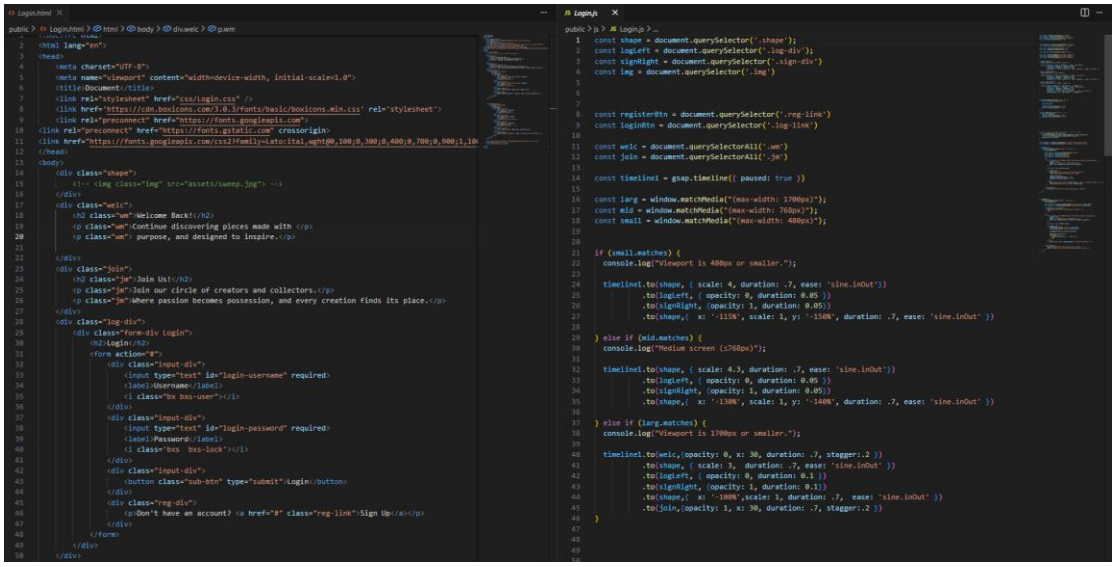












```
<!-- Login.html -->
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link rel="stylesheet" href="css/Login.css" />
    <link href='https://cdn.boxicons.com/3.0.3/fonts/basic/boxicons.min.css' rel='stylesheet'>
    <link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Lato:ital,wght@0,100;0,300;0,400;0,700;0,900;1,10
</head>
<body>
    <div class="shape">
        <!-- <img class="img" src="assets/sweep.jpg"> -->
    </div>
    <div class="welc">
        <h2 class="wm">Welcome Back!</h2>
        <p class="wm">Continue discovering pieces made with </p>
        <p class="wm"> purpose, and designed to inspire.</p>

    </div>
    <div class="join">
        <h2 class="jm">Join Us!</h2>
        <p class="jm">Join our circle of creators and collectors.</p>
        <p class="jm">Where passion becomes possession, and every creation finds its place.</p>
    </div>
    <div class="log-div">
        <div class="form-div Login">
            <h2>Login</h2>
            <form action="#">
                <div class="input-div">
                    <input type="text" id="login-username" required>
                    <label>Username</label>
                    <i class="bx bxs-user"></i>
                </div>
                <div class="input-div">
                    <input type="text" id="login-password" required>
                    <label>Password</label>
                    <i class='bxs  bxs-lock'></i>
                </div>
                <div class="input-div">
                    <button class="sub-btn" type="submit">Login</button>
                </div>
                <div class="reg-div">
                    <p>Don't have an account? <a href="#" class="reg-link">Sign Up</a></p>
                </div>
            </form>
        </div>
    </div>
</body>
```

```
// Login.js
const shape = document.querySelector('.shape');
const logLeft = document.querySelector('.log-div');
const signRight = document.querySelector('.sign-div')
const img = document.querySelector('.img')



const registerBtn = document.querySelector('.reg-link')
const loginBtn = document.querySelector('.log-link')

const welc = document.querySelectorAll('.wm')
const join = document.querySelectorAll('.jm')

const timeline1 = gsap.timeline({ paused: true })

const larg = window.matchMedia("(max-width: 1700px)");
const mid = window.matchMedia("(max-width: 768px)");
const small = window.matchMedia("(max-width: 480px)");


if (small.matches) {
  console.log("Viewport is 480px or smaller.");

  timeline1.to(shape, { scale: 4, duration: .7, ease: 'sine.inOut'})
           .to(logLeft, { opacity: 0, duration: 0.05 })
           .to(signRight, {opacity: 1, duration: 0.05})
           .to(shape,{  x: '-115%', scale: 1, y: '-150%', duration: .7, ease: 'sine.inOut' })

} else if (mid.matches) {
  console.log("Medium screen (≤768px)");

  timeline1.to(shape, { scale: 4.3, duration: .7, ease: 'sine.inOut'})
           .to(logLeft, { opacity: 0, duration: 0.05 })
           .to(signRight, {opacity: 1, duration: 0.05})
           .to(shape,{  x: '-130%', scale: 1, y: '-140%', duration: .7, ease: 'sine.inOut' })

} else if (larg.matches) {
  console.log("Viewport is 1700px or smaller.");

  timeline1.to(welc,{opacity: 0, x: 30, duration: .7, stagger:.2 })
           .to(shape, { scale: 3,  duration: .7, ease: 'sine.inOut' })
           .to(logLeft, { opacity: 0, duration: 0.1 })
           .to(signRight, {opacity: 1, duration: 0.1})
           .to(shape,{  x: '-100%',scale: 1, duration: .7,  ease: 'sine.inOut' })
           .to(join,{opacity: 1, x: 30, duration: .7, stagger:.2 })
}
```